

UXOXO

A Universal UI Language:
the expressible, the addressable, and the idealized interface

A formal monograph

June 28, 2026

Contents

1	Elements and Commands: the UI–DSL Correspondence	1
1.1	Preliminaries: the objects to be joined	2
Part A.	One signature, two dual carriers	4
1.2	The shared signature	4
1.3	Commands, and the Free/Cofree pairing	7
Part B.	The element–command correspondence	9
1.4	The UI element, defined through the DSL	9
1.5	The behavioural bijection: every state-change is a command	10
1.6	Attributes are an option set; the overlay is the cascade	12
Part C.	What the functional companion buys	13
1.7	Routing is a traversal; dispatch is ordered choice	13
1.8	Rendering is comonadic; serialization is a prism	14
1.9	One expressiveness ladder, four readings	16
1.10	The learned UI is a learned program	17
Part D.	Calibration: where functional programming is not optimal	19
1.11	Identity lives off the functor	19
1.12	Local high-frequency mutation defeats value semantics	21
1.13	Observer graphs exceed trees	22
1.14	The theory fixes what, not when	23
1.15	The stratification: the right tool per layer	24
Part E.	Synthesis	25
1.16	The correspondence, assembled	25
1.17	Crosswalk: one object across the companions	27
2	The Expressible and the Idealized UI	28
2.1	Preliminaries and standing assumptions	29
Part A.	The expressible UI	30
2.2	Workspaces and elements	30
2.3	The authoring system	31
2.4	The runtime system, and finiteness	32
2.5	The UI-state space	33
2.6	Generative completeness and expressibility	34
2.7	Edit mode	35
2.8	The four faces of expressibility	36
Part B.	Preference and the ideal	38
2.9	Contexts as a fibration	38

2.10	Preference and its utility representation	39
2.11	The ideal policy and the optimal transversal	40
2.12	The invariant lattice, precisely	41
2.13	Personalization as a learned invariant	42
Part C.	Learning the ideal	43
2.14	The behavioural observation model	43
2.15	The usage model and the Bayes update	44
2.16	Consistency: the current weights reach the ideal	45
2.17	The predictive law and the MAP policy	46
Part D.	The agent and the loop	47
2.18	The value-driven agent	47
2.19	The closed loop and non-thrashing	48
2.20	What is computable	49
Part E.	Synthesis and limits	50
2.21	Synthesis	50
2.22	What is free, what is assumed	51
2.23	The twelve truths, located	53

Chapter 1

Elements and Commands: the UI–DSL Correspondence

*The command companion makes a command a node of the free monad $\text{Free}\Omega A$ — program-as-data, a first-class value with three legs (parse, compose, interpret). This note shows that the user-interface tree is the dual of that object: the stateful UI is the cofree comonad $\text{Cofree}\Omega \text{St}$ over the same signature, an element-shaped tree annotated with state and behaviour. From this single fact the correspondence the informal account demands falls out rigorously. Structurally, an element’s type is a constructor of Ω , with its child-profile the constructor’s arity — one namespace worn three ways: UI type, command keyword, opcode tag. Behaviourally, an element is a Mealy node, and a command is a choice of event; the event loop is the $\text{Free}/\text{Cofree}$ pairing that feeds the choice to the responder, and we prove this pairing is exactly the framework’s runtime step. Closing the loop, the interpretation map is a monoid isomorphism between closed commands (modulo behaviour) and the reachable UI state-changes — the precise form of “every state and event is a command, and conversely.” An element’s attributes are an option set, its overlay the options companion’s $\oplus$; a component with unfilled children is a template. We then say what the functional companion buys — routing as a **Traversable** descent, rendering as the comonadic **extend**, serialization as the parsing prism, the learned personalization agent as a closed program — and, with equal care, where it is not optimal: node identity lives off the functor, local high-frequency mutation defeats value semantics, observer graphs exceed trees, and the equational theory fixes what a program computes but not when. The boundary is exact: pure tree $\rightarrow$ free/cofree; finite sharing $\rightarrow$ a coalgebra; cycles, identity, or churn $\rightarrow$ the imperative.*

1.1 Preliminaries: the objects to be joined

New in this section. signature functor F ; types $\mathbb{T}$, profile $\text{prof}(\tau)$, state space $\text{St}(\tau)$; UI tree μF ; free monad $\text{Free}\Omega A$ (command) and cofree comonad $\text{Cofree}F C$ (annotated tree); Mealy handler handle_τ , events Ev ; option set O with algebra $(\oplus, \sim, \text{diff})$.

We recall, compactly but precisely, the four objects this note ties together, fixing notation and standing assumptions. Proofs of the recalled facts are in the cited companions; everything new is proved here.

The element-shape functor. A UI is a finite, typed, ordered tree of elements. The shape of that tree is governed by a single **signature functor**.

Definition 1.1 (Signature). Let $\mathbb{T}$ be a finite set of **element types**. The **signature functor** F is the polynomial functor

$$F X = \sum_{\tau \in \mathbb{T}} \left(\text{Attr}_\tau \times X^{\text{prof}(\tau)} \right),$$

one summand per type, carrying that type’s attribute record Attr_τ and a tuple of child-positions indexed by the type’s **profile** $\text{prof}(\tau)$ (a finite, possibly empty, position set). F is built from constants, $+$, $\times$, and exponentials by finite arity, hence *polynomial*; we assume it **Traversable** (Assumption 1.2). Each type’s **state space** $\text{St}(\tau)$ is a finite set assembled from atoms by product ($\times$), sum ($+$), and least fixed point (μ).

Assumption 1.2 (Standing assumptions). $\mathbb{T}$, every Attr_τ , and every $\text{St}(\tau)$ are finite; F is a polynomial **Traversable** functor; trees are finite and well-founded; and the concrete-syntax layout of each constructor (Proposition 1.24) is uniquely decodable. We work up to identity-isomorphism (renaming nodes changes no observable), except in Section 1.11, which is precisely about identity.

The UI tree. The bare shape is the initial algebra μF — the finite F -trees, i.e. the well-formed workspaces $\mathcal{T}$ of the editor chapter. An editor acts on μF by the structural primitives $\mathcal{O}_\top = \{\text{mint}, \text{attach}, \text{detach}, \text{delete}, \text{relabel}\}$, and we recall its central property; the **runtime** layer instead drives node state via a Mealy handler per type, $\text{handle}_\tau : \text{St}(\tau) \times \text{Ev} \rightarrow \text{St}(\tau)$, over a finite event alphabet Ev .

Theorem 1.3 (Generative completeness; recalled). *The structural primitives are universal: from the seed $W_\emptyset$ they reach exactly $\mathcal{T}$, and $(\mathcal{T}, \rightarrow)$ is strongly connected. Behaviourally, for each type the runtime relation generated by handle_τ reaches all of $\text{St}(\tau)$ from its initial state (with edit mode supplying any state a handler omits). Consequently every valid UI configuration is reachable, and the joint system is finite, hence decidable to analyse.*

The command. The command companion fixes the **command signature** Ω — one operator per name, of arity its parameter profile — and the free monad

$$\text{Free}\Omega A = \mu X. (A + \Omega X),$$

the **free** $\langle \Omega, A \rangle$ carrier: a **command is a node of** $\text{Free}\Omega A$, first-class because it is a value in a **Monad**. A **closed** command ($A = \emptyset$) lies in $\text{Free}\Omega \emptyset \cong \mu \Omega$, a runnable program. Three maps act on this one term: **parse** : $\Sigma^* \rightarrow \text{Free}\Omega A$ (build, an unfold), **compose** : $\text{Free}\Omega \emptyset \rightarrow \Sigma^*$ (serialize, a fold), and $\llbracket - \rrbracket : \text{Free}\Omega \emptyset \rightarrow T(R)$ (interpret, a fold into an effect monad T).

Options. An **option** is a key–value pair $o = (k(o), v(o))$ with $k(o)$ in a key space; an **option set** O has injective key map, so $k(\cdot) : O \cong K(O)$ and $O[k]$ denotes the option at key k . The companion’s algebra $(\oplus, \sim, \text{diff})$ gives: the **precedence overlay** $O_1 \oplus O_2$ (keep all keys, O_2 wins on overlap), an associative monoid with identity $\emptyset$; **agreement** $O_1 \sim O_2$ (identical on shared keys), a reflexive–symmetric tolerance (not transitive); and the projected **difference** diff_π . Overlay is commutative exactly on agreeing sets.

The functional carriers. We use the companion’s carriers **maybe**, **result**, **pair/kv.pair** (the environment comonad), **view/producer** (lazy sequences), and the two that organise this note: $\text{free}\langle F, A \rangle = \text{Free} F A$ (a **Monad**, program-as-data) and its dual $\text{cofree}\langle F, C \rangle = \text{Cofree} F C$ (a **Comonad**, an annotated tree). The protocols invoked are **Functor**, **Applicative**, **Selective**, **Monad**, **Traversable**, **Foldable**, **Alternative**, and **Comonad**.

Remark 1.4 (The thesis in one line). **Free** and **Cofree** over the *same* F are categorical duals. The command companion develops the **Free** side; the UI framework, unknowingly, develops the **Cofree** side. Naming that duality is the whole content of the correspondence: an element and a command are one signature read backwards and forwards.

1.2 The shared signature

New in this section. the identification $\Omega = F$: one constructor read as UI *type*, command *keyword*, *opcode*, and parameter/child *profile* (its arity).

The first move is to observe that the element types and the command names are not two alphabets to be related but one alphabet seen twice.

Definition 1.5 (Identification of signatures). Identify the command signature with the element signature, $\Omega = F$, by sending each element type $\tau \in \mathbb{T}$ to the command constructor of the same name and arity $\text{prof}(\tau)$. Under this identification a constructor is at once: an element **type** (a node kind of the UI tree), a command **name** (a production keyword of the DSL), and — via the opcode companion — an **opcode** (a machine word). The constructor's arity is read three ways: the type's **child-profile**, the production's right-hand side, and the command's **parameter profile**.

Proposition 1.6 (Types are constructors). *The identification $\Omega = F$ is a bijection on constructors that preserves arity, and it is forced once one asks that building an element and naming its command use the same data. In particular $\mu F = \mu \Omega$ as sets of finite trees: a workspace shape is a closed first-order command term, and conversely.*

Proof. Both signatures are sums indexed by a name set with a finite position tuple per summand; the identification matches names and, by Definition 1.5, positions, so it is an arity-preserving bijection of polynomial functors. Initial algebras of equal functors are equal, giving $\mu F = \mu \Omega$. The forcing is the observation that a node's children and a command's parameters are the same finite indexed family: to build the node is to supply exactly the data the command names. $\square$

Remark 1.7 (Three coordinates name one thing). The opcode companion encodes the constructor *tag*; the parse/compose pair relates the surface *keyword* to that tag; so a single element/command is pinned by three coordinates — keyword (concrete syntax), constructor (abstract syntax), opcode (machine word) — with **parse**, **compose** the left edge and opcode assignment the right. The element's *type* is the same constructor, making the UI node the fourth reading of the one tag. The polarity pairs, product attributes, and hierarchies the opcode companion encodes (PAIR, PROD, TREE) are exactly the $+$, $\times$, μ of the state algebra St : flat attributes are a Π -product (PROD), a containment hierarchy a dependent Σ -product (TREE).

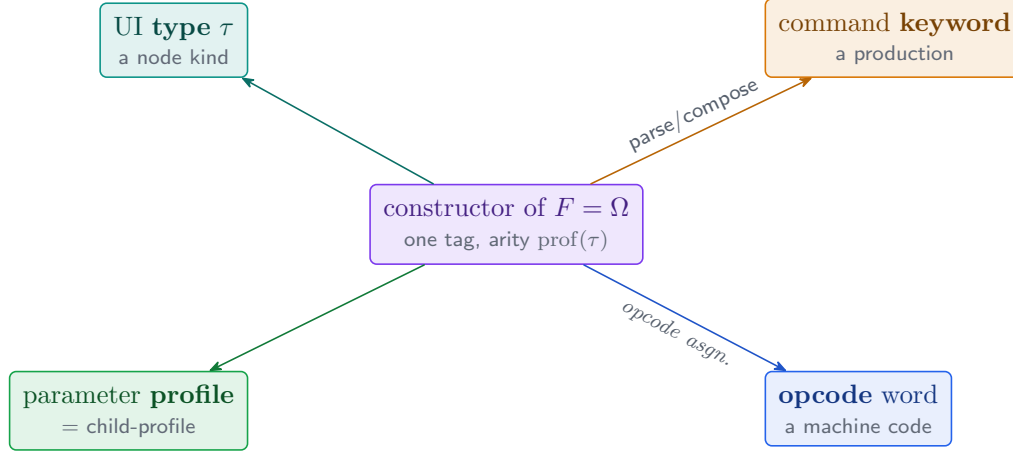


Figure 1.1: **Four faces of one constructor.** A single tag of the shared signature $\Omega = F$ is read as a UI type, a DSL keyword, an opcode, and a parameter/child profile; **parse/compose** is the keyword $\leftrightarrow$ tag edge and opcode assignment the tag $\leftrightarrow$ word edge (Definition 1.5 and Remark 1.7).

The bare tree μF has no state. The running UI annotates every node with its current state (and, below, its behaviour). That annotated tree is a cofree comonad.

Definition 1.8 (The stateful UI tree). The **stateful UI tree** is the carrier

$$U = \mu X. (\text{St} \times F X),$$

the finite F -trees in which every node v additionally carries an annotation $\sigma(v) \in \text{St}(\tau(v))$. It is the well-founded fragment of the cofree comonad $\text{Cofree } F \text{ St}$, and carries the comonad operations

$$\text{extract} : U \rightarrow \text{St} \quad (\text{the root's annotation}), \quad \text{duplicate} : U \rightarrow \mu X. (U \times F X),$$

where **duplicate** relabels each node v by the entire sub-tree rooted at v ; **extend** $g = F(\dots) \circ \text{relabel}$ applies $g : U \rightarrow B$ at every node to its own sub-tree, **extend** $g : U \rightarrow \mu X. (B \times F X)$.

Proposition 1.9 (U is a comonad; the laws are the UI's locality). *(extract, duplicate) satisfy the comonad laws $\text{extract} \circ \text{duplicate} = \text{id}$, $F(\text{extract}) \circ \text{duplicate} = \text{id}$ (after the evident reassociation), and coassociativity. Reading them in UI terms: (i) the root sub-tree's root is the root (extract after duplicate), so a node's annotation is recoverable from its context; (ii) annotating each node with its own singleton context is the identity; (iii) contexts of contexts agree with contexts (coassociativity), so a context-dependent pass may be staged. Hence any rendering or layout whose output at a node depends on that node's sub-tree is a well-defined extend.*

Proof. These are the standard cofree-comonad laws for the functor $X \mapsto \text{St} \times F X$, which hold for any F ; on the well-founded (μ) fragment the relabelings terminate, so the operations are total. The UI readings are the laws transcribed under Definition 1.8: **duplicate** places at v the sub-tree at v , whose **extract** is $\sigma(v)$, giving (i); **extend extract** = **id** gives (ii); coassociativity is the equality of the two ways to form contexts-of-contexts, (iii). $\square$

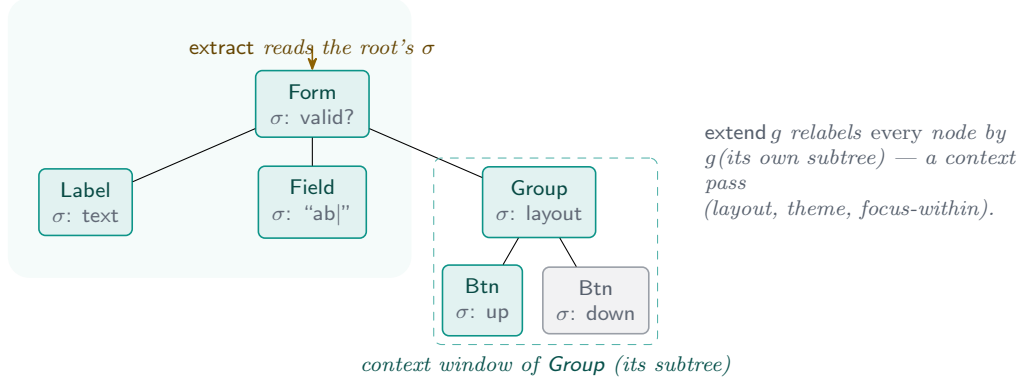


Figure 1.2: **The stateful UI is a cofree comonad.** An element-shaped tree (F -shape) with a state annotation σ at every node: the carrier $U = \mu X.(\text{St} \times FX)$ of Definition 1.8. **extract** reads a node's own state; **extend** recomputes each node from its subtree context (the dashed window) — the comonadic form of context-dependent layout and rendering (Propositions 1.9 and 1.23).

Definition 1.10 (Behavioural annotation; the element as a Mealy node). Annotate each node additionally with its handler. Currying the Mealy map presents a node of type τ as a **response product**

$$\widehat{\text{handle}}_\tau : \text{St}(\tau) \longrightarrow \prod_{e \in \text{Ev}} (\text{St}(\tau) + \{\perp\}), \quad \widehat{\text{handle}}_\tau(s)(e) = \begin{cases} \text{handle}_\tau(s, e), & e \in E(\tau), \\ \perp, & \text{otherwise,} \end{cases}$$

i.e. for each current state a chosen next-state *for every event* (absent where unhandled). The **behavioural UI** is the cofree tree $\widehat{U} = \text{Cofree}F(\text{St} \times \prod_e (\text{St} + \{\perp\}))$ carrying both state and response product at each node. An **element** is one node of $\widehat{U}$: a type, an attribute record, a state, and a response product.

The response product is the cofree side's payload. The next section exhibits its dual — a command — and the operation that pairs them.

1.3 Commands, and the Free/Cofree pairing

New in this section. the located command (p, e) and command program; the pairing run and the interpreter $\llbracket - \rrbracket$.

A command is a node of $\text{Free}\Omega A$ (Section 1.1). Where a handler node *provides* a response for every event (a product), a command *selects* one (a sum). Product pairs with sum; the pairing is application, and it is the event loop.

Definition 1.11 (Located command; event selection). An **atomic located command** is a pair (p, e) of a path p to a node of U and an event $e \in \text{Ev}$ (with any payload folded into e); equivalently it is the **choice**, at the addressed node, of one coordinate of that node’s response product. A **command program** is a node of $\text{Free}\Omega A$ whose constructors are located commands — a free-monad sequence/branch of selections; a closed program lies in $\text{Free}\Omega \emptyset$. Authoring commands ($\text{mint}, \dots, \text{relabel}$) are the located commands whose effect is structural rather than on σ .

Definition 1.12 (The pairing / run). Define $\text{run} : \text{Free}\Omega \emptyset \times U \rightarrow U$ by structural recursion on the program. On an atomic located command (p, e) at the node $v = U@p$,

$$\text{run}((p, e), U) = U[v := \widehat{\text{handle}}_{\tau(v)}(\sigma(v))(e)] \quad \text{if } e \in E(\tau(v)), \quad \text{else } U,$$

i.e. replace the addressed node’s annotation by the selected response, leaving shape and all other annotations fixed; on a sequence $c_1; c_2$, $\text{run}(c_1; c_2, U) = \text{run}(c_2, \text{run}(c_1, U))$; on an authoring command, apply the corresponding structural primitive. Equivalently $\text{run} = \llbracket - \rrbracket$ with the effect monad T the State monad over U , $R = \mathbf{1}$.

Theorem 1.13 (The pairing is the runtime step). *For an atomic located command (p, e) routing to node v , $\text{run}((p, e), -)$ equals the framework’s runtime step $\xRightarrow{e}$ that routes e to v ; for an authoring command it equals the corresponding editor step $\rightarrow$. Hence on programs, run is the sequential composition of runtime and authoring steps, and the interpretation $\llbracket - \rrbracket$ of a closed command is the joint UI transition it denotes.*

Proof. The runtime step, by definition, routes e to the addressed node v and updates its state by $\sigma(v) \mapsto \widehat{\text{handle}}_{\tau(v)}(\sigma(v), e)$ when $e \in E(\tau(v))$ (and is the identity otherwise), changing nothing else; this is exactly the right side of Definition 1.12 after uncurrying $\widehat{\text{handle}}$. Authoring constructors are interpreted as their structural primitives by construction. Sequencing in the free monad is interpreted as composition in the State monad (the behaviour chapter’s $f_{c_2} \bullet f_{c_1}$), which is the composition of the corresponding steps. Therefore $\text{run} = \llbracket - \rrbracket$ agrees step-for-step with the joint transition relation. $\square$

Remark 1.14 (Why this is a *pairing*). The content is the duality *product pairs with sum*: a handler node is a product over events (provide a response to each), a command is a sum over events (choose one), and run is the canonical evaluation $(\prod_e R_e) \times (\sum_e \mathbf{1}) \rightarrow R$ applied node-wise. Extended along the recursion, this is the standard pairing of a **cofree**-comonad of handlers with a **free**-monad of commands: the UI is the comonad that *responds*, the command is the monad that *drives*, and the event loop is their pairing. The framework’s two transition systems (runtime and authoring) are the two halves of this one operation — state-moves and shape-moves — because $\Omega = F$ carries both.

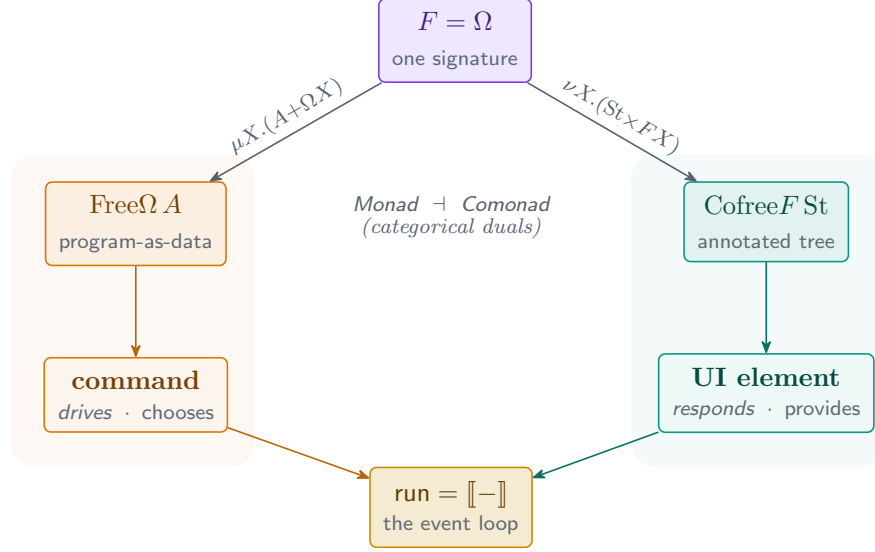


Figure 1.3: **One signature, two dual carriers.** The command companion develops the free monad $\text{Free}\Omega A$ (it *drives*: a program that chooses); the UI framework develops the cofree comonad $\text{Cofree}F\text{St}$ (it *responds*: a tree that provides). Both are built over the same $F = \Omega$, and the event loop $\text{run} = []$ is their pairing (Theorem 1.13).

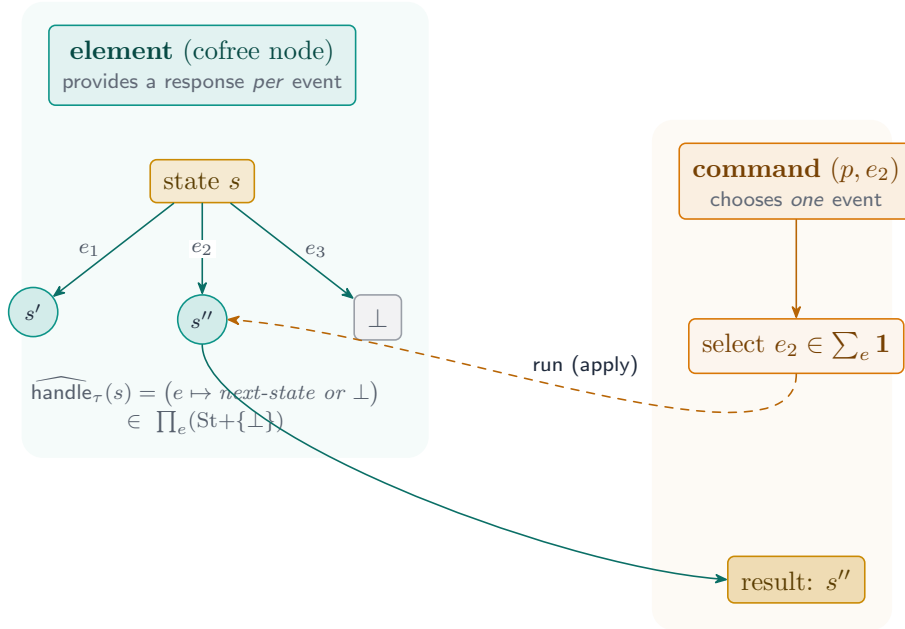


Figure 1.4: **Product pairs with sum: the pairing is application.** A handler node is a *product* over events (a next-state for each); a command is a *choice* of one event (a summand). The run feeds the choice to the product and reads off the selected response — exactly the runtime step (Theorem 1.13 and Remark 1.14).

1.4 The UI element, defined through the DSL

New in this section. the *UI element* $\delta = (\tau, \text{attrs}, \sigma, \widehat{\text{handle}}_\tau)$ in four faces; *component* (composite element) and *template* (a node of $\text{Free}\Omega A$ with holes).

We can now give the element its definition — not as a primitive, but through its four relations to the surrounding framework.

Definition 1.15 (UI element; component). A **UI element** is a node of the behavioural UI tree $\widehat{U}$ (Definition 1.10), i.e. a tuple

$$\delta = (\tau, \text{attrs}, \sigma, \widehat{\text{handle}}_\tau)$$

read through four faces:

- (1) **(DSL face)** τ is a *constructor of Ω* of arity $\text{prof}(\tau)$ — the element’s command name / production / opcode, with child-profile the parameter profile (Definition 1.5);
- (2) **(option face)** attrs is an *option set* keyed by attribute name — equally a command *binding* (Section 1.6);
- (3) **(state face)** $\sigma \in \text{St}(\tau)$ is a point of the $\times / + / \mu$ state algebra;
- (4) **(behaviour face)** $\widehat{\text{handle}}_\tau$ is a *Mealy node* — the cofree annotation, and the *interpret-leg* of the element’s commands (Theorem 1.13).

A **component** is a composite element: an internal node of $\widehat{U}$, hence a sub-term of the command tree. A component with one or more unfilled child positions is a *template* over Ω (a node of $\text{Free}\Omega A$ with holes A); filling them is instantiation $t \gg= (u \triangleleft d)$ (Section 1.9).

Remark 1.16 (Atomic vs. composite, once more). An **atomic element** is a leaf: a nullary constructor of Ω , a *bare command*. A **composite element** is an internal node: a constructor of positive arity, a command carrying sub-commands. Nesting is composition; there is no separate “container” primitive, exactly as there is no separate composite command — both are just internal nodes of the one tree (Proposition 1.6).

1.5 The behavioural bijection: every state-change is a command

New in this section. closed commands $\mathcal{C}^0 = \text{Free}\Omega \emptyset$; behavioural equivalence $\approx$; the state-change monoid $\text{Trans}(\widehat{U})$.

The informal demand is that “all elements map to a DSL command for all states/events, and conversely.” Taken naively — one element, one command — it is a category error: an element admits many commands (one per event, plus authoring). The correct statement separates the two axes and is exact on each.

Theorem 1.17 (Behavioural bijection). *Let $\mathcal{C}^0 = \text{Free}\Omega \emptyset$ be the closed commands, and let $c \approx c'$ mean $\llbracket c \rrbracket = \llbracket c' \rrbracket$ as transformations of $\widehat{U}$. Let $\text{Trans}(\widehat{U})$ be the monoid (under composition) of UI state-changes generated by the runtime events and authoring primitives. Then $\llbracket - \rrbracket$ descends to a **monoid isomorphism***

$$\llbracket - \rrbracket : \mathcal{C}^0 / \approx \xrightarrow{\cong} \text{Trans}(\widehat{U}).$$

Equivalently: every UI state-change is realised by a closed command, distinct state-changes by inequivalent commands, and command sequencing is transition composition. This is the precise sense of “every state and event is a command, and conversely.”

Proof. Well-defined and injective. By construction $\approx$ is the kernel of $\llbracket - \rrbracket$, so the induced map on $\approx$ -classes is well-defined and injective.

Homomorphism. The behaviour chapter interprets a program $c_1; c_2$ as $\llbracket c_2 \rrbracket \bullet \llbracket c_1 \rrbracket$, composition in the State monad; thus $\llbracket c_1; c_2 \rrbracket = \llbracket c_2 \rrbracket \circ \llbracket c_1 \rrbracket$, and $\llbracket - \rrbracket$ carries free-monad sequencing to composition, with the empty program to id.

Surjectivity (the “conversely”). $\text{Trans}(\widehat{U})$ is generated by two families: each runtime event e routed to each node, and each authoring primitive. By Theorem 1.13 the former is $\llbracket (p, e) \rrbracket$ for the located command addressing that node, and the latter is $\llbracket c \rrbracket$ for the corresponding authoring constructor; so every *generator* of $\text{Trans}(\widehat{U})$ lies in the image. A monoid homomorphism whose image contains a generating set is onto the generated monoid. Hence $\llbracket - \rrbracket$ is surjective, and with injectivity an isomorphism.

Coverage of all states. That the generators actually reach every configuration — so $\text{Trans}(\widehat{U})$ acts transitively on the valid states — is generative completeness (Theorem 1.3): authoring reaches every shape and is strongly connected, and the per-type runtime reaches every state. Thus not only is each *move* a command, but the commands suffice to reach *every* state from any other. $\square$

Corollary 1.18 (The two correct readings). *The correspondence holds on two distinct axes, and only these:*

- (1) structural — element **types** $\leftrightarrow$ command **constructors**, an arity-preserving bijection (Proposition 1.6);
- (2) behavioural — UI **state-changes** $\leftrightarrow$ closed **commands** modulo behaviour, a monoid isomorphism (Theorem 1.17).

The naive “element $\leftrightarrow$ command” conflates a constructor with a closed term over it; resolved into these two axes, “everything is a command” is literally true.

Proof. Immediate from the cited results; the conflation is the type error of equating an element type (a generator) with an individual program built from it. $\square$

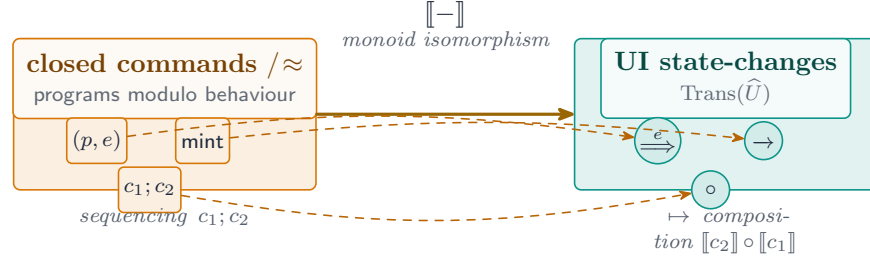


Figure 1.5: **The behavioural bijection (Theorem 1.17).** Interpretation $\llbracket - \rrbracket$ sends closed commands modulo behaviour *isomorphically* onto the reachable UI state-changes: generators (events, edits) map to generators, sequencing to composition. “Every state and event is a command, and conversely” — surjectivity is generative completeness, injectivity is the definition of $\approx$.

1.6 Attributes are an option set; the overlay is the cascade

New in this section. attribute record = command binding = option set; `setAttr`, `setStyle` as overlay $\oplus$; the cascade (left $\oplus$ -fold) and the reconciler diff_π .

The option face of Definition 1.15 is an equality, not an analogy.

Theorem 1.19 (Attributes/binding/options coincide). *For any element, the following are the same object: its **attribute record**, the **binding** of its constructor’s command, and an **option set** keyed by attribute/parameter name. Under this identification:*

- (1) `setAttr` and `setStyle` are the precedence overlay: applying a patch P is $\text{attrs} \mapsto \text{attrs} \oplus P$;
- (2) a **style cascade** (defaults, then theme, then inline) is the left fold $A_{\text{def}} \oplus A_{\text{theme}} \oplus A_{\text{inline}}$, well-defined and order-consistent (“later wins”) since (option sets, $\oplus, \emptyset$) is a monoid;
- (3) two attribute sources are co-applicable in any order iff they agree ($\sim$) — the commutativity-iff-agreement law, identical to the modifier-commutativity law for command behaviour;
- (4) the change set the reconciler computes between UI states s, s' is $\text{diff}_\pi(\text{attrs}(s), \text{attrs}(s'))$, with π selecting declared value, default, or value-or-default.

Proof. A command binding assigns to each parameter (a unique key) a value-or-absence $\beta_c(a) \in D_a \cup \{\perp\}$, with β_c injective on the profile; this is exactly an option set keyed by the profile (synthesis chapter). An attribute record is the same finite key $\rightarrow$ value map keyed by attribute name; the three views differ only in the name of the index set. The four clauses are then the options companion’s laws transported verbatim: (1) is the definition of $\oplus$ as “ O_2 overrides O_1 on shared keys”; (2) is associativity with identity $\emptyset$; (3) is the proposition that $O_1 \oplus O_2 = O_2 \oplus O_1 \iff O_1 \sim O_2$, which the behaviour chapter shows coincides with pairwise commutativity of the corresponding modifiers; (4) is the definition of diff_π . $\square$

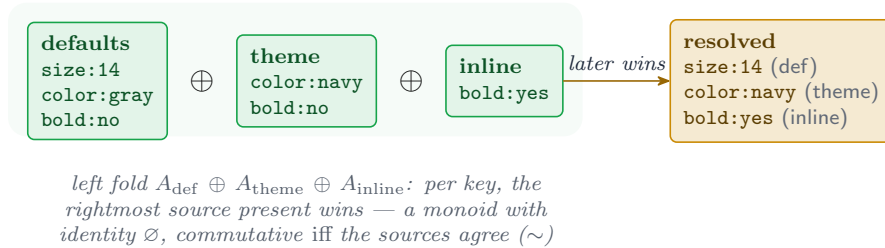


Figure 1.6: **The style cascade is the option overlay $\oplus$.** Defaults, theme, and inline attributes are option sets; cascading them is the left fold of the precedence overlay (“later wins”), keyed per attribute. This is literally the options companion’s monoid and the command’s sequential override (Theorem 1.19 and Remark 1.20).

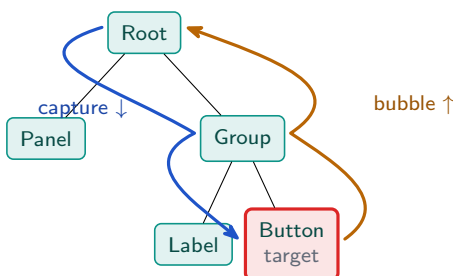
Remark 1.20 (One resolution device everywhere). “Later wins” (style cascade, sequential override), “leftmost wins” (committed event dispatch, PEG choice), and “highest precedence wins” (opcode tactic) are the single device of a total order making a nondeterministic combine deterministic, instantiated at apply-time, parse-time, and encode-time. Attribute cascading is this device on the option monoid; the next part shows event dispatch is the same device on the parser’s **Alternative**.

1.7 Routing is a traversal; dispatch is ordered choice

New in this section. routing as the **Traversable** descent **traverse**; dispatch as left-biased **alt** (**Alternative**); the capture and bubble phases.

Proposition 1.21 (Routing as a **Traversable** descent). *Hit-testing and event dispatch are one **Traversable** traversal of F with **Alternative** selecting the consuming node. Concretely, routing an event is the effectful descent **traverse** over the node’s children that (i) by the disjoint-region geometry visits the unique child whose region contains the pointer (or, for keyboard events, follows the focus path), and (ii) at each level offers the local handler via left-biased **alt**: the first ancestor-or-node whose handler is defined on the event consumes it. The two dispatch phases — capture ($\text{root} \rightarrow \text{target}$) and bubble ($\text{target} \rightarrow \text{root}$) — are the pre-order and post-order of the same traversal.*

Proof. F is **Traversable** (Assumption 1.2), so **traverse** threads an effect across a constructor’s children in a fixed order; geometry makes the pointer-containing child unique (the editor’s disjoint-sibling condition), so the descent is deterministic. Left-biased **alt** is, by the parser companion, exactly committed ordered choice: **alt** $p\ q$ runs p , and only on its failure q ; taking p to be “the deeper handler” and q “the shallower” realises “first defined handler wins,” i.e. event capture; reversing the bias gives bubbling. Pre-/post-order are the two threadings of one **Traversable** structure. $\square$



*one **Traversable** descent; **Alternative** (left-biased) picks the first handler that consumes — the committed choice of PEG and override*

Figure 1.7: **Routing is a traversal; dispatch is ordered choice (Proposition 1.21).** Geometry/focus fixes a unique $\text{root} \rightarrow \text{target}$ path. The two dispatch phases are the pre-order (capture, $\downarrow$) and post-order (bubble, $\uparrow$) of one **Traversable** descent; “first handler wins” is left-biased **Alternative**.

Remark 1.22 (Dispatch precedence is the resolution device). Thus event precedence (which handler fires) is not UI-specific machinery but the **Alternative** monoid’s **alt/aempty** — the same committed choice as PEG parsing and the dual, at run-time, of sequential override (Remark 1.20). That **result** is *not* **Alternative** is why an error-reporting dispatcher must elect an explicit merge rather than inherit a free “no handler.”

1.8 Rendering is comonadic; serialization is a prism

New in this section. rendering as the comonadic `extend`; the serialization prism `compose = cata[print]` and `parse = compose-1` on L ; the canonical command surface.

Proposition 1.23 (Rendering as `extend`). *A render or layout pass whose output at a node depends on that node’s sub-tree (intrinsic sizing, theme inheritance, focus-within) is the comonadic `extend` $g : U \rightarrow U'$ for $g : U \rightarrow B$ reading the local context; `extract` reads a node’s own value. An incremental render is `extend` restricted to a dirty sub-comonad: nodes whose context is unchanged keep their previous output by the comonad law `extend extract = id`.*

Proof. By Proposition 1.9, a context-dependent assignment is a well-defined `extend`; `extract` is the local read. If g depends only on an unchanged sub-context at v , then g at v is unchanged, so re-running `extend` g there reproduces the prior value — formally, on the clean sub-comonad g factors through `extract`, where `extend extract = id`. $\square$

Proposition 1.24 (Serialization/addressing as the parsing prism). *The UI tree round-trips through the DSL: `compose = cata[print] :  $\mu F \rightarrow \Sigma^*$` is a total injection, and `parse = compose-1 :  $\Sigma^* \rightarrow \mu F$` its partial inverse on $L = \text{im}(\text{compose})$, satisfying the prism laws `parse(compose  $t$ ) =  $t$` and $s \in L \Rightarrow \text{compose}(\text{parse } s) = s$. Consequently every element and every UI state has a **canonical command surface** naming it, and conversely each surface in L denotes a unique UI.*

Proof. F is a polynomial **Traversable** functor with a uniquely decodable layout (Assumption 1.2); the parsing companion then gives `compose` as the unique fold, total and (by decodability) injective, and `parse` as its anamorphic inverse on the image, with the prism laws. Applying `compose` to a sub-tree yields the surface naming that element; applying it to the state-decorated tree (the decorated grammar) names a state. Injectivity gives uniqueness in both directions. $\square$

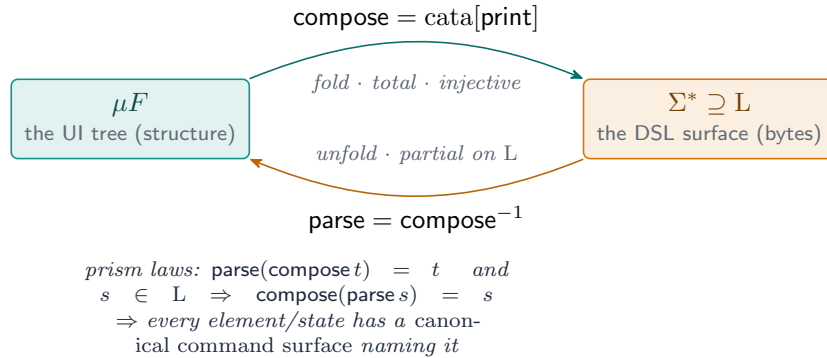


Figure 1.8: **Serialization is a prism; the UI is addressable (Proposition 1.24).** `compose` folds a UI tree to its DSL surface (total, injective); `parse` unfolds it back on the recognized language L . The round-trip laws make “name this element/state by a command” well-defined — the bridge that lets a closed command in Theorem 1.17 denote a state-change concretely.

Remark 1.25 (Addressability is the bridge to Part B). Proposition 1.24 is what lets a *closed command* in Theorem 1.17 *name* a state-change concretely: parse the surface to a command term,

run it by the pairing. The grammar is the UI's declarative image; the pairing is its operational one; the prism is the isomorphism between them on L .

1.9 One expressiveness ladder, four readings

New in this section. the expressiveness ladder $\text{Free}_{\text{Ap}} \subseteq \text{Free}_{\text{Sel}} \subseteq \text{Free}$ (applicative $\subseteq$ selective $\subseteq$ monadic).

Proposition 1.26 ($\text{Free}_{\text{Ap}} \subseteq \text{Free}_{\text{Sel}} \subseteq \text{Free}$, read four ways). *The free-construction ladder grades, by exactly the $\times, +, \mu$ strata, four notions at once:*

	Free_{Ap}	Free_{Sel}	Free
<i>parser</i>	<i>fixed shape</i>	<i>static guard</i>	<i>value-driven shape</i>
<i>template</i>	<i>fill-in-the-blanks</i>	<i>conditional section</i>	<i>recursive/data-driven</i>
<i>command</i>	<i>independent args</i>	<i>flag gates a branch</i>	<i>arg chooses sub-command</i>
<i>UI element</i>	<i>static layout</i>	<i>conditional child</i>	<i>data-driven subtree</i>

The inclusions are strict and the same in all four columns: no value-inspection (applicative), static branch only (selective), a resolved value chooses the substructure (monadic, the μ case).

Proof. The ladder is the parser/template grading of the companions; Definition 1.5 makes the UI element’s child-shape an Ω -term, so a child position fixed in advance is applicative, a child present-or-absent under a static guard is the selective **when**, and a child sub-tree chosen by a resolved value is the monadic $\gg$ over μ . Strictness is inherited from the companions: a value that selects shape is not expressible without $\gg$. $\square$

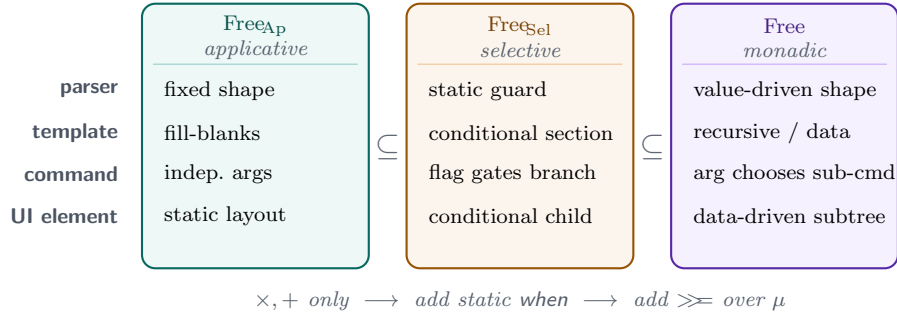


Figure 1.9: **One ladder, four readings (Proposition 1.26).** The free-construction chain $\text{Free}_{\text{Ap}} \subseteq \text{Free}_{\text{Sel}} \subseteq \text{Free}$ grades parser, template, command, and UI expressiveness identically: no value-inspection (applicative), a static branch (selective), a resolved value choosing the substructure (monadic, the μ case).

Remark 1.27 (Where each UI idiom sits). A static form is applicative; “show this panel only if logged in,” with both states pre-built, is selective; a tree/list whose node count is a runtime value, or a wizard whose next step depends on an answer, is monadic. The selective rung is the corpus’s guarded action $\gamma(w, e) = \text{when}$ at term level; the monadic rung is where genuine data-dependence — and the cost discussion of Part D — begins.

1.10 The learned UI is a learned program

New in this section. the capability-permitted sub-signature Ω_κ ; the glamour $g \in \text{Free}\Omega_\kappa \emptyset$; the agent (fairy) as the interpreter $\llbracket - \rrbracket$.

The personalization layer now reads in command terms, tightening its safety guarantees to statements about $\llbracket - \rrbracket$.

Proposition 1.28 (Glamour as closed program; agent as interpreter). *Let $\Omega_\kappa \subseteq \Omega$ be the sub-signature of capability-permitted constructors. The preemptive agent’s **glamour** is a closed program $g \in \text{Free}\Omega_\kappa \emptyset$; the agent is the free-monad interpreter $\llbracket - \rrbracket$ run as a pre-pass on the default tree. Then:*

- (1) (safety) $\llbracket g \rrbracket$ preserves every shipped invariant — schema, no-dangling, protection — because each Ω_κ -constructor does and invariant-preservation composes; so $\llbracket g \rrbracket(\hat{U})$ is valid;
- (2) (reversibility) $\llbracket g \rrbracket$ is undoable and reset-to-default is reachable, by strong connectivity (Theorem 1.3);
- (3) (optimality target) compiling the learned utility’s per-context optimum into g makes $\llbracket g \rrbracket$ drive the default to that optimum, when it is κ -reachable.

Hence “personalize the UI” is “synthesize and run a closed command”; the learned model emits a program, and the lattice of invariants bounds it.

Proof. A glamour is a guarded rewrite by κ -permitted operations, i.e. a finite located-command program over Ω_κ , hence a node of $\text{Free}\Omega_\kappa \emptyset$; running it is $\llbracket - \rrbracket$ (Definition 1.12). (1) Each permitted constructor is a step of the invariant-restricted editor, preserving schema and (with the global-reset convention) no-dangling; preservation of a meet of invariants is the conjunction of preservations, so the composite program preserves all of them. (2) The unrestricted editor is strongly connected (Theorem 1.3), so from $\llbracket g \rrbracket(\hat{U})$ the default — and any prior state — is reachable; undo is the reverse program. (3) The optimum is a valid state (Theorem 1.3 coverage); if it is reachable by κ -permitted steps, the synthesized g is a reaching program and $\llbracket g \rrbracket$ lands there. $\square$

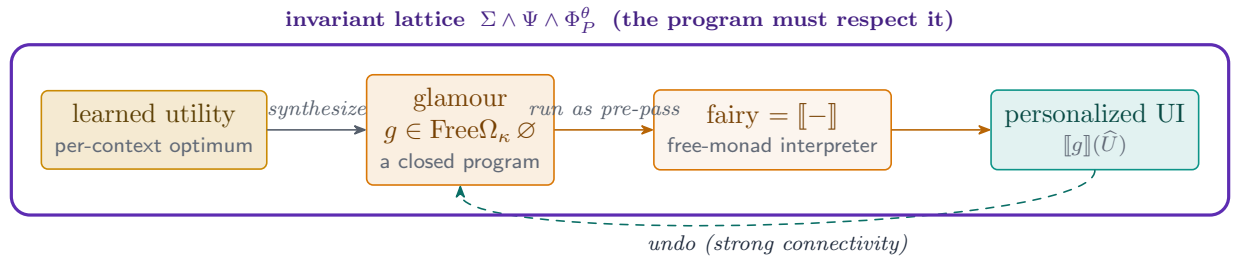


Figure 1.10: **The learned UI is a learned program (Proposition 1.28).** Personalization compiles the learned utility into a closed command $g \in \text{Free}\Omega_\kappa \emptyset$ over capability-permitted constructors; the agent runs it by $\llbracket - \rrbracket$. Safety is that $\llbracket g \rrbracket$ stays inside the invariant lattice; reversibility is strong connectivity. “Personalize” = “synthesize and run a command.”

Remark 1.29 (The whole stack in one sentence). Reading bottom to top: a *type* is a constructor; an *element* is a cofree node over it; a *state-change* is a closed command paired against that node; a *UI* is the cofree tree; *personalization* is a synthesized free program run against it; and the invariant lattice is the predicate the program must respect. One signature $\Omega = F$ threads all six.

Part D. Calibration: where functional programming is not optimal

The functional reading is exact and buys the results above, but it is optimal only on a precise fragment. Four propositions mark the boundary; the part closes by locating it.

1.11 Identity lives off the functor

New in this section. the runtime state $\varrho = (U, \text{globals})$ with pointer map $\text{globals} : \{\dots\} \rightarrow N$; the integrity constraint Ψ ; identities $N \subseteq \mathbb{I}$.

Proposition 1.30 (The runtime state is not a Free/Cofree value). *The cofree tree U is determined by shape and annotation up to isomorphism and carries no node identity. But the runtime globals — focus, hover, capture, drag, selection — are references to specific nodes, and the no-dangling invariant Ψ is the integrity constraint that these references point into the live document. Therefore the full runtime state is not a value of any Free/Cofree carrier; it is the pair*

$$\varrho = (U, \text{globals}), \quad \text{globals} : \{\text{focus, hover, capture, drag, selection}\} \rightarrow N,$$

a cofree tree plus a partial pointer assignment into its node-identity set $N \subseteq \mathbb{I}$. Stability of these pointers across mutations is a property no identity-quotiented (pure value) representation provides.

Proof. Free and Cofree are functors of F alone; their elements are equivalence classes under relabeling of positions, so two trees differing only by node names are equal, i.e. identity is quotiented away. The globals, however, must denote *the same* node before and after an unrelated edit elsewhere; this is data not recoverable from the shape-and-annotation class, hence external to the carrier. The pairing with $\text{globals} : \cdot \rightarrow N$ is exactly the extra datum, and Ψ is the well-formedness of that partial map. Renaming-invariance of the carrier and pointer-stability of the globals are contradictory unless identity is tracked separately. $\square$

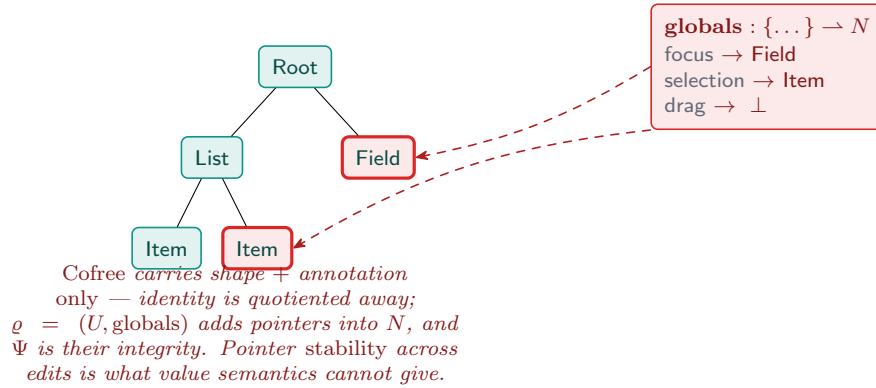


Figure 1.11: **Identity lives off the functor (Proposition 1.30).** The cofree tree U knows shape and state but not *which* node is which; focus, selection, and drag are references into the identity set N . The runtime state is the pair $\varrho = (U, \text{globals})$, and Ψ is the no-dangling integrity of those pointers — a non-polynomial side constraint.

Remark 1.31 (Off the hierarchy, as foretold). This is the corpus’s observation that identity integrity sits off the Chomsky hierarchy, now as the functional boundary: a regular/polynomial functor

captures shape; key-and-reference identity is a non-polynomial side constraint. The clean fix is an imperative node table with stable handles, which the value semantics of Free/Cofree deliberately lacks.

1.12 Local high-frequency mutation defeats value semantics

New in this section. update costs $\Theta(\text{depth})$ (pure spine copy) and $\Theta(\text{size})$ (extend) vs. $O(1)$ (imperative); the frequency $\times$ locality parameter.

Proposition 1.32 (Update costs). *A single runtime event changes one node’s annotation. In the pure carrier U this requires rebuilding the path from the root to that node — $\Theta(\text{depth})$ fresh nodes per event — and a comonadic extend recomputes every node, $\Theta(\text{size})$. An in-place representation with the stable handles of Proposition 1.30 performs the same local update in $O(1)$ amortized and re-render in $O(\#\text{affected})$. Hence on the high-frequency, spatially-local events that dominate interaction (typing, dragging, hovering), value semantics is asymptotically suboptimal; the governing parameter is the frequency $\times$ locality of mutation, not the data model’s correctness.*

Proof. Persistent update of an immutable μ -tree must allocate a new spine from the mutated node to the root to preserve sharing of the untouched subtrees, which is $\Theta(\text{depth})$ nodes; this is the standard cost of functional update. A global extend visits every node by definition. An imperative table mutates the target cell in place and marks a bounded dirty set, giving the stated bounds. For a stream of m events each touching $O(1)$ nodes, the pure cost is $\Theta(m \cdot \text{depth})$ versus imperative $\Theta(m)$ plus affected re-render. $\square$

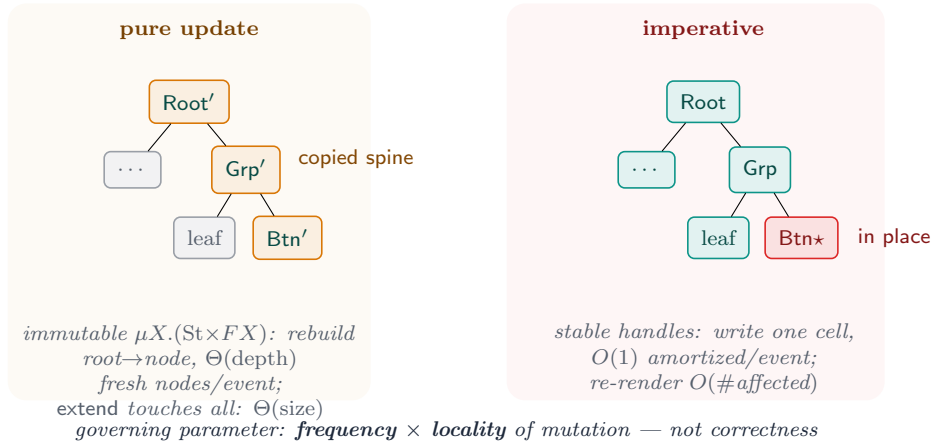


Figure 1.12: **Local high-frequency mutation defeats value semantics (Proposition 1.32).** One event changes one node. Persistent update must copy the spine to the root ($\Theta(\text{depth})$, and extend is $\Theta(\text{size})$); an imperative table with stable handles mutates one cell in $O(1)$. Value semantics still wins for snapshots, undo, and history — the trade is directional.

Remark 1.33 (Where value semantics still wins). The same persistence that costs on local writes *buys* cheap snapshotting, undo, structural sharing, and time-travel — precisely the operations the personalization layer (Proposition 1.28) and the editor’s history want. The trade is real and directional: pure for infrequent global moves and history, imperative for frequent local ones. A practical UI uses both, separated by exactly this boundary.

1.13 Observer graphs exceed trees

New in this section. the sharing coalgebra $\text{Struct}(F)$ (a finite DAG) and the cyclic/coalgebraic regime νF .

Proposition 1.34 (Trees, sharing DAGs, cyclic graphs). *Data-binding, two-way bindings, and observers make the dependency structure a graph, whereas μF and $\text{Cofree}F$ are trees. The corpus already lifts μF to $\text{Struct}(F)$ — a finite coalgebra with sharing edges — to express back-references and shared partials; this captures acyclic sharing (a DAG). Genuine cycles (mutual data-binding, observer loops) are not finite F -trees and not $\text{Struct}(F)$ DAGs; they require a coalgebraic (νF) or imperative (pointer-graph) representation with an explicit update/fixpoint discipline.*

Proof. A finite tree has a unique path to each node and no node reachable from itself; an observer cycle has a node depending on its own output, so it is not modeled by μF . $\text{Struct}(F)$ adds sharing as a finite coalgebra structure, representing common sub-structure (a DAG) but still terminating unfolds; a cycle makes the unfold non-terminating, leaving the $\text{Struct}(F)$ fragment. The coalgebraic νF or an explicit pointer graph with a scheduling/fixpoint rule is then required to give the loop a meaning. $\square$

1.14 The theory fixes what, not when

New in this section. the effect monad $T(R)$; the value/schedule distinction (what a program computes vs. when).

Remark 1.35 (Effects: value versus schedule). The interpretation $\llbracket - \rrbracket$ lands in an effect monad $T(R)$, and the monad/comonad laws fix the *value* a program computes and the *equalities* between programs. They do *not* fix the *schedule*: real-time deadlines, animation timing, frame pacing, input coalescing, and the interleaving of concurrent effects are outside the equational theory. Two programs equal in T may differ in latency or perceived smoothness. The algebra is the contract for *correctness*; timing is an operational concern the functional reading deliberately abstracts, and where those constraints bind, an effect-scheduling layer outside the laws is required.

1.15 The stratification: the right tool per layer

New in this section. the three representation regimes: pure tree $\rightarrow$ Free/Cofree; sharing DAG $\rightarrow$ Struct(F); identity/cycles/churn $\rightarrow$ imperative.

Proposition 1.36 (Representation boundary). *The four observations partition the UI's structure into three regimes, each with a fitting representation:*

- (1) pure finite tree — the shape μF and the stateful tree $U = \text{Cofree } F \text{ St}$, with infrequent or global moves, history, and serialization: the **functional** carriers Free/Cofree are optimal (Parts A–C);
- (2) finite acyclic sharing — back-references and shared partials: the **coalgebraic** Struct(F) (a finite F -coalgebra) is the right representation;
- (3) identity-critical, cyclic, or high-frequency — focus/selection pointers (Proposition 1.30), observer graphs (Proposition 1.34), and per-frame local mutation (Proposition 1.32): a mutable, identity-bearing, **imperative** representation is optimal.

The boundaries are exact and are read off the structure itself — acyclicity, sharing, identity-dependence, mutation frequency — not chosen by taste.

Proof. Each clause is the cited result: Propositions 1.23 and 1.24 place pure trees in (1); the corpus's Struct(F) lift places acyclic sharing in (2) (Proposition 1.34); and Propositions 1.30, 1.32 and 1.34 place identity, cycles, and churn in (3). The criteria are structural predicates (presence of cycles, of sharing, of identity references, the frequency $\times$ locality of writes), so the assignment is determined by the structure. $\square$

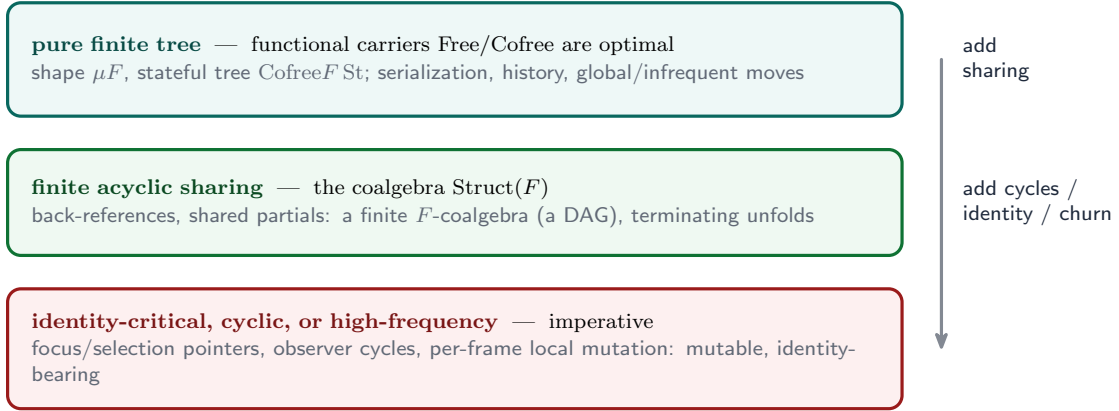


Figure 1.13: **The stratification: the right tool per layer (Proposition 1.36).** The boundary is read off the structure itself — acyclicity, sharing, identity-dependence, mutation frequency. Pure trees $\rightarrow$ free/cofree; acyclic sharing $\rightarrow$ the coalgebra Struct(F); cycles, identity, or churn $\rightarrow$ the imperative. The functional reading of Parts A–C is exact and optimal on the top zone, and not beyond it.

1.16 The correspondence, assembled

Theorem 1.37 (The UI–DSL correspondence). *Under Assumption 1.2 and generative completeness (Theorem 1.3), for the shared signature $\Omega = F$:*

- (1) (dual carriers) *commands are the free monad $\text{Free}\Omega A$ and the stateful UI is the cofree comonad $\text{Cofree}F \text{ St}$ over the same F , paired by the event loop $\text{run} = \llbracket - \rrbracket$, which is the runtime/authoring step (Theorem 1.13);*
- (2) (structure) *element types are command constructors, an arity-preserving bijection, with three further faces keyword/opcode/profile (Proposition 1.6 and Remark 1.7);*
- (3) (behaviour) *closed commands modulo behaviour are isomorphic, as a monoid, to the reachable UI state-changes — every state and event is a command and conversely (Theorem 1.17);*
- (4) (data) *an element’s attributes are an option set, its overlay the cascade $\oplus$, its diff the reconciler (Theorem 1.19); and a component with holes is a template (Definition 1.15);*
- (5) (functional support) *routing is a **Traversable** descent with **Alternative** dispatch, rendering the comonadic **extend**, serialization the parsing prism, and personalization a synthesized closed program (Propositions 1.21, 1.23, 1.24 and 1.28);*
- (6) (boundary) *this functional reading is optimal exactly on the pure finite-tree fragment; acyclic sharing wants the coalgebra $\text{Struct}(F)$, and identity/cycles/churn want the imperative (Proposition 1.36).*

Proof. Each clause is the cited result; together they are the assembled statement that a UI element and a DSL command are one signature $\Omega = F$ read as a responding comonad and a driving monad respectively. $\square$

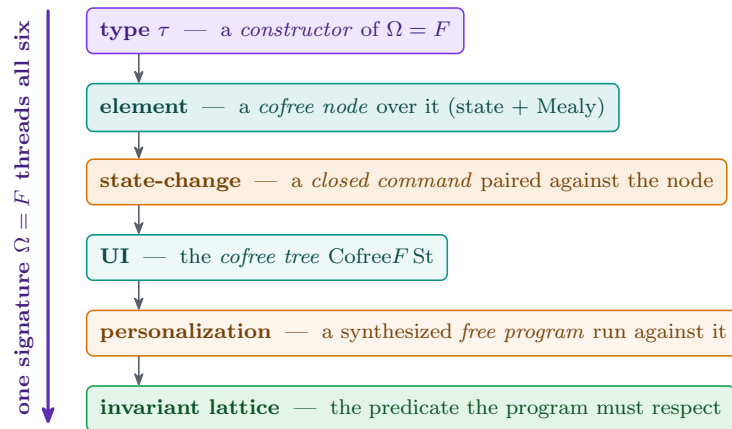


Figure 1.14: **The whole stack, threaded by one signature (Theorem 1.37 and Remark 1.29).** Bottom to top, each layer is a reading of the same $\Omega = F$: a type is a constructor; an element a cofree node; a state-change a closed command paired against it; the UI the cofree tree; personalization a synthesized free program; the lattice the predicate it must respect.

Remark 1.38 (What is necessary, what is contingent). *Necessary* — forced by the framework: the identification $\Omega = F$ (Proposition 1.6); the free/cofree duality and its pairing (Theorem 1.13); the behavioural isomorphism (Theorem 1.17); and attributes-as-options (Theorem 1.19). These hold however the system is built. *Contingent* — a representation choice: whether a given layer is realised in the functional carriers, the coalgebra, or the imperative. The correspondence is a theorem; the functional implementation is an option, optimal precisely where Proposition 1.36 says and not elsewhere. The single thread through both is the signature: meaning enters once, when a constructor is declared, and is read forwards as a command and backwards as an element.

1.17 Crosswalk: one object across the companions

UI notion	Command / companion incarnation	Where
element type τ	constructor of $\Omega = F$; keyword; opcode tag; child-profile = arity	Definition 1.5 and Proposition 1.6
atomic element (leaf)	nullary constructor; bare command	Remark 1.16
composite element	internal node of $\text{Cofree}F$; command carrying sub-commands	Remark 1.16
component with a hole	template over Ω ; node of $\text{Free}\Omega A$	Definition 1.15
stateful UI tree	cofree comonad $\text{Cofree}F \text{ St} = \mu X.(\text{St} \times FX)$	Definition 1.8
element (with behaviour)	Mealy node = response product = interpret-leg	Definition 1.10
event at a node	located command (p, e) ; choice of one response coordinate	Definition 1.11
runtime / authoring step	the $\text{Free}/\text{Cofree}$ pairing $\text{run} = \llbracket - \rrbracket$	Theorem 1.13
all state-changes	closed commands $\text{Free}\Omega \emptyset$ modulo $\approx$ (monoid iso)	Theorem 1.17
attributes / style	option set; $\text{setAttr}/\text{cascade} = \oplus$; reconcile = diff_π	Theorem 1.19
event dispatch	Traversable descent + Alternative ordered choice	Proposition 1.21
render / layout	comonadic extend ; local read extract	Proposition 1.23
serialize / address	parsing prism compose / parse on μF	Proposition 1.24
static/conditional/data-driven UI	$\text{Free}_{\text{Ap}} \subseteq \text{Free}_{\text{Sel}} \subseteq \text{Free}$ ladder	Proposition 1.26
personalization glamour	closed program $\text{Free}\Omega_\kappa \emptyset$; agent = $\llbracket - \rrbracket$	Proposition 1.28
focus/selection/drag	pointers into N ; $\varrho = (U, \text{globals})$ — off the functor	Proposition 1.30
back-reference / shared partial	coalgebra $\text{Struct}(F)$ (acyclic sharing)	Proposition 1.34
observer cycle / per-frame churn	imperative, identity-bearing representation	Proposition 1.36

Chapter 2

The Expressible and the Idealized UI

A user interface is modelled as a finite, typed, ordered tree of elements carrying two transition systems — an authoring system that moves its shape and a runtime system that moves its state. We first make precise the sense in which such a UI can express every valid state: generative completeness, from which a regular tree grammar, an instruction algebra, a direct-manipulation editor, and decidability follow as four faces of one reachable set. We then add a user. A preference over the finite state space is, by a representation theorem, a utility; its per-context maximizers form an ideal policy whose graph is a transversal of a context fibration, and enforcing it is one more meet in a lattice of invariants — a construction we show is a meet-semilattice homomorphism into sub-transition-systems. A usage model learns this preference by Bayesian updating over a finite hypothesis class from the trajectories of both transition systems; we prove posterior consistency (the current weights converge to the ideal weights) under realizability, identifiability, and exploration. A value-driven agent compiles the inferred ideal into a provably safe and reversible pre-pass, and we prove that, with a confidence gate as the sole damping parameter, the closed loop switches finitely often and converges. The idealized UI is therefore the expressible UI under a learned invariant, cast preemptively — no new power, one new constraint.

2.1 Preliminaries and standing assumptions

New in this section. the size bound B and the identity-isomorphism quotient; labelled transition system (LTS) and its induced sub-LTS $(\mathcal{S}, \rightarrow) \upharpoonright_\phi$.

We fix notation and two finiteness conventions used throughout.

A **labelled transition system** (LTS) is a pair $(\mathcal{S}, \rightarrow)$ with $\rightarrow \subseteq \mathcal{S} \times \mathcal{S}$ (we suppress action labels when irrelevant). For $\phi \subseteq \mathcal{S}$ the **induced sub-LTS** is $(\mathcal{S}, \rightarrow) \upharpoonright_\phi := (\phi, \rightarrow \cap (\phi \times \phi))$: keep the states in ϕ and the edges with both endpoints in ϕ . We write $\rightarrow^*$ for reflexive–transitive closure and call $(\mathcal{S}, \rightarrow)$ **strongly connected** if $s \rightarrow^* s'$ for all s, s' .

Assumption 2.1 (Finite types and bounded size). The type alphabet $\mathbb{T}$ and the event alphabet Ev are finite, and every type’s state space $\text{St}(\tau)$ is finite. We fix a **size bound** $B \in \mathbb{N}$ and consider only workspaces with $|N| \leq B$.

The size bound is what every *finiteness* and *optimization* claim rests on; the purely *structural* reachability results (Theorem 2.7) hold without it. A bound is no loss of generality for a deployed UI: a screen, a DOM, a scene graph are all finite.

Assumption 2.2 (Identities are immaterial). Two workspaces are identified when related by a bijection of their node sets preserving root, parent, child-order, and labels. All counting and optimization are over isomorphism classes; “the UI” never depends on the names in $\mathbb{I}$.

This quotient is what makes the otherwise-unbounded identity domain $\mathbb{I}$ (over which key and reference constraints are not even regular) drop out of the finite analysis: renaming nodes changes no observable.

2.2 Workspaces and elements

New in this section. workspace $W = (N, r, p, \chi, \ell)$, document $\text{Doc}(W)$, descendant orders $\triangleleft, \triangleleft^*, \triangleleft^+$; well-formed set $\mathcal{T}$ and schema-valid $\mathcal{T}_\Sigma$; element $\delta = (\tau, A, S, E, \rho)$, state algebra $\text{St}(\tau)$, region **reg**; atomic vs. composite.

Definition 2.3 (Workspace). A **workspace** is a tuple $W = (N, r, p, \chi, \ell)$ with N a finite node set, $r \in N$ a root, $p : N \rightarrow N$ a partial parent map, $\chi(n)$ a duplicate-free ordering of $p^{-1}(n)$, and ℓ a labelling by elements (Definition 2.4). The **document** is the r -reachable part $\text{Doc}(W) := \{n \in N : r \triangleleft^* n\}$, where $\triangleleft$ is “ $= p(\cdot)$ ” and $\triangleleft^*$ its reflexive–transitive closure; nodes of $N \setminus \text{Doc}(W)$ head **detached fragments**. W is **well-formed** when

WF1. p is a partial function (at most one parent per node);

WF2. $\chi(n)$ enumerates exactly $p^{-1}(n)$ without repetition;

WF3. $\triangleleft^+$ is irreflexive (acyclic; (N, p) is a forest);

WF4. $r \notin \text{dom}(p)$ and $\text{Doc}(W)$ is the r -reachable set;

WF5. ℓ is total and each label is well-typed (Definition 2.4).

$\mathcal{T}$ denotes the well-formed workspaces and, for a content model (schema) Σ fixing which types may parent which, $\mathcal{T}_\Sigma := \{W \in \mathcal{T} : \Sigma(W)\}$. The trivial schema $\Sigma \equiv \top$ gives the *ideal* case.

Definition 2.4 (Element). A type $\tau \in \mathbb{T}$ denotes a finite state space $\text{St}(\tau)$ built from atomic state sets by product (*concurrent*, $|A \times B| = |A||B|$), sum (*exclusive*, $|A + B| = |A| + |B|$), and least fixed point (*nesting*, $\mu X. F(X)$). An **element** is a tuple $\delta = (\tau, A, S, E, \rho)$: a type tag τ ; an attribute record A carrying at least a region $\text{reg}(\delta) \subseteq \mathbb{R}^d$; the state space $S = \text{St}(\tau)$; a handled alphabet $E \subseteq \text{Ev}$; and a policy ρ comprising a partial **handler** $\text{handle}_\tau : S \times \text{Ev} \rightarrow S$ with $\text{dom}(\text{handle}_\tau) \subseteq S \times E$ (a Mealy machine), a render contribution, and a **capability set** $\kappa(n) \subseteq \{\text{remove, reparent, reorder, restyle, retype, addChild, rmChild}\}$. A node with no children is an **atomic element** (a *leaf*); an internal node is a **composite element**.

Remark 2.5 (Composition is the tree). There is no separate “composite” primitive: a composite element *is* an internal node, and nesting *is* composition. Geometric coherence ($\text{reg}(n) \subseteq \text{reg}(p(n))$ and disjoint sibling regions) makes hit-testing (Definition 2.9) total and deterministic.

2.3 The authoring system

New in this section. the editor LTS $\mathcal{E}$; primitive operations $\mathcal{O}_\top = \{\text{mint}, \text{attach}, \text{detach}, \text{delete}, \text{relabel}\}$ and derived **move**; the authoring relation $\rightarrow$.

Definition 2.6 (Editor). An **editor** is an LTS $\mathcal{E} = (\mathcal{S}, \rightarrow)$, $\mathcal{S} \subseteq \mathcal{T}$, whose steps are the **primitive operations**, each a partial function on workspaces defined only when its result is again well-formed:

- (1) $\text{mint}(\delta) \Rightarrow n$: add a fresh parentless node labelled δ (a singleton fragment); always defined.
- (2) $\text{attach}(n, q, k)$: when n is a fragment root, q lies outside n 's subtree, $0 \leq k \leq |\chi(q)|$; set $p(n) = q$, insert n at index k of $\chi(q)$.
- (3) $\text{detach}(n)$: when $n \in \text{dom}(p)$, $n \neq r$; remove n from $\chi(p(n))$ and unset $p(n)$ (the subtree leaves with it).
- (4) $\text{delete}(n)$: when n is a fragment root, $n \neq r$; remove n and its subtree from N .
- (5) $\text{relabel}(n, \delta')$: set $\ell(n) = \delta'$ (refined into **setType**, **setAttr**, **setStyle** to touch one component).

Write $\text{move}(n, q, k) := \text{detach}(n); \text{attach}(n, q, k)$ and $\mathcal{O}_\top = \{\text{mint}, \text{attach}, \text{detach}, \text{delete}, \text{relabel}\}$. A step is written $W \rightarrow W'$; the relation $\rightarrow$ is the union of the primitive-operation graphs.

Theorem 2.7 (Structural universality). *From the seed $W_\emptyset$ (a lone root r), the set reachable under $\mathcal{O}_\top$ is exactly $\mathcal{T}$, and $(\mathcal{T}, \rightarrow)$ is strongly connected. Under Assumption 2.1 the same holds within $\mathcal{T}^{\leq \mathbf{B}}$.*

Proof. Soundness. Each precondition returns the result to WF1–WF5: **mint** adds an isolated labelled node; **attach**'s no-cycle precondition preserves the forest and updates χ consistently; **detach** splits off a subtree; **delete** drops a tree; **relabel** touches only ℓ . Hence $\rightarrow$ never leaves $\mathcal{T}$.

Completeness. Fix a target $W^* \in \mathcal{T}$. **mint** each node of W^* with its target label; then visit the nodes of each tree of W^* in *preorder* and **attach** every non-root beneath its target parent at its target index. Preorder guarantees the parent is already present and rooted, so each **attach** precondition holds; detached fragments are rebuilt the same way. The result is W^* . Every intermediate workspace has at most $|N(W^*)|$ nodes, so if $|N(W^*)| \leq \mathbf{B}$ the build stays within $\mathcal{T}^{\leq \mathbf{B}}$.

Strong connectivity. From any W , repeatedly **detach** a leaf and **delete** it (sizes only decrease), reaching $W_\emptyset$; composing the teardown of W with the build of W' gives $W \rightarrow^* W'$. Sizes never exceed $\max(|N(W)|, |N(W')|)$, so connectivity holds within $\mathcal{T}^{\leq \mathbf{B}}$ as well. $\square$

Proposition 2.8 (Primitivity — no sealed compounds). $\rightarrow^*$ is generated by single primitive steps: $W \rightarrow^* W'$ iff there is a finite word $o_1 \cdots o_m \in \mathcal{O}_\top^*$ carrying W to W' . Consequently every reachable transformation factors through the exposed primitives, and any derived operation (e.g. **move** = **detach**; **attach**) is definitionally such a word. There is no transformation reachable by some compound **doXY** that its constituents **doX**, **doY** cannot also achieve.

Proof. By Definition 2.6 the one-step relation $\rightarrow$ is the union of the primitive-operation graphs; $\rightarrow^*$ is its reflexive–transitive closure, i.e. exactly the pairs joined by a finite word in $\mathcal{O}_\top$. A compound is by definition a word in $\mathcal{O}_\top$, so its reachable set is contained in $\rightarrow^*$, which by Theorem 2.7 is all of $\mathcal{T}$ (resp. $\mathcal{T}^{\leq \mathbf{B}}$). Hence no compound reaches outside the primitives' closure. $\square$

2.4 The runtime system, and finiteness

New in this section. runtime state ϱ and its space $\text{RT}(W)$; the no-dangling invariant Ψ ; hit, route, handle and the runtime relation $\Longrightarrow$.

Definition 2.9 (Runtime state). For W , a **runtime state** is

$$\varrho = (\sigma, \text{focus}, \text{hover}, \text{capture}, \text{drag}, \text{selection}).$$

It carries a state map $\sigma : N \rightarrow \bigcup_n \text{St}(\ell(n))$ with $\sigma(n) \in \text{St}(\ell(n))$; pointer/keyboard globals $\text{focus}, \text{hover}, \text{capture} \in N \cup \{\perp\}$; a drag slot $\text{drag} \in (N \times \text{payload}) \cup \{\perp\}$; and a bounded selection $\subseteq N$. The space is $\text{RT}(W)$. The **no-dangling** invariant is

$$\begin{aligned} \Psi(W, \varrho) : & \iff \text{focus}, \text{hover}, \text{capture} \in \text{Doc}(W) \cup \{\perp\} \wedge \\ & \text{drag} \in (\text{Doc}(W) \times \text{payload}) \cup \{\perp\} \wedge \text{selection} \subseteq \text{Doc}(W). \end{aligned}$$

The **hit** of a point x is the deepest node whose region contains x (unique, by disjoint siblings); routing sends a pointer event to capture or the hit, a key event to focus. A **runtime step** $\varrho \xrightarrow{e} \varrho'$ (shape fixed) updates the routed node's state by $\sigma(n) \mapsto \text{handle}_{\tau(n)}(\sigma(n), e)$ when $e \in E(n)$, with the guarded global updates, and preserves Ψ automatically. Authoring steps that remove a node reset any global pointing into it to $\perp$, re-establishing Ψ .

Theorem 2.10 (Finiteness and decidability of the per-workspace runtime). *For fixed W with each $\text{St}(\ell(n))$ finite,*

$$\text{RT}(W) \cong \prod_{n \in N} \text{St}(\ell(n)) \times (N \cup \{\perp\})^3 \times ((N \times \text{payload}) \cup \{\perp\}) \times \{U \subseteq N : |U| \leq k\}$$

is finite, so $(\text{RT}(W), \Longrightarrow)$ is a finite transition system and reachability, equivalence, and LTL/CTL model-checking over it are decidable.

Proof. Each factor is finite (N finite, St finite), so the product is finite; $\Longrightarrow$ is a relation on a finite set, and the cited decision problems are decidable for finite transition systems (standard). $\square$

2.5 The UI-state space

New in this section. the UI state $s = (W, \varrho)$ and the (finite) UI-state space $\mathbb{U}_{\Sigma}^{\leq B}$.

Definition 2.11 (UI state, UI-state space). A **UI state** is a pair $s = (W, \varrho)$ with $W \in \mathcal{T}_{\Sigma}$, $|N| \leq B$, $\varrho \in \text{RT}(W)$, and $\Psi(W, \varrho)$. The **UI-state space** $\mathbb{U}_{\Sigma}^{\leq B}$ is the set of all such pairs, taken up to the isomorphism of Assumption 2.2.

Proposition 2.12 (The state space is finite). $\mathbb{U}_{\Sigma}^{\leq B}$ is finite and nonempty.

Proof. Nonempty: $(W_{\emptyset}, \varrho_0) \in \mathbb{U}_{\Sigma}^{\leq B}$. Finite: by Assumption 2.2 a workspace of size $\leq B$ is an isomorphism class of a finite rooted ordered forest on $\leq B$ nodes labelled from the finite alphabet $\mathbb{T}$ (with finite attribute records, themselves drawn from finite ranges under Assumption 2.1); there are finitely many such classes. For each, $\text{RT}(W)$ is finite by Theorem 2.10, and Ψ cuts a subset. A finite union of finite sets is finite. $\square$

Remark 2.13 (Validity is built in, not patched on). “Some configurations are invalid — e.g. two radio buttons filled at once” is handled at the level of the type algebra, not by an external filter. A radio group of n is the sum type $\sum_n 1$, so “exactly one filled” holds by construction: “two filled” is not a point of $\text{St}(\tau)$. The general discipline is to absorb impossible combinations into the type, or else carve them out by a constraint or the schema Σ ; the no-dangling invariant Ψ removes the remaining invalid global states. $\mathbb{U}_{\Sigma}^{\leq B}$ is precisely what survives.

2.6 Generative completeness and expressibility

New in this section. generative completeness (structural + behavioural) and *expressibility*; initial state $s_0(\tau)$; the reachable set $\text{Reach}(W_\emptyset, \rightarrow_\Sigma)$.

Write $s_0(\tau) \in \text{St}(\tau)$ for a type's initial state and $\text{Reach}(W_\emptyset, \rightarrow_\Sigma)$ for the workspaces reachable from the seed under the Σ -restricted primitives.

Definition 2.14 (Generative completeness; expressibility). A UI is **generatively complete** relative to Σ when: (a) *structural* — $(\mathcal{T}_\Sigma, \rightarrow)$ is universal: $\text{Reach}(W_\emptyset, \rightarrow_\Sigma) = \mathcal{T}_\Sigma$ and the graph is strongly connected; (b) *behavioural* — for each τ , the runtime system $(\text{St}(\tau), \Longrightarrow)$ generated by handle_τ reaches all of $\text{St}(\tau)$ from $s_0(\tau)$. It is **expressible** when, in addition, its authoring moves are *primitive* in the sense of Proposition 2.8.

Clause (a) is Theorem 2.7. Clause (b) is not automatic: a monotone handler (a counter that only counts up) violates it. The repair is edit mode (Section 2.7): a state no handler reaches at runtime can be *set* as a property.

Remark 2.15 (The name). *Generative*, not *Turing*: the runtime is deliberately finite-state (Theorem 2.10), hence sub-Turing; completeness here is *surjectivity of the generators* (they reach every configuration), which is what keeps analysis decidable. The word is apt because the property reads two ways that Section 2.8 shows coincide — the operations *generate* the reachable set, the grammar *generates* the language.

2.7 Edit mode

New in this section. the mode $m \in \{\text{use}, \text{edit}\}$; the capability set $\kappa(n)$; the mode escape.

Definition 2.16 (Edit mode). Augment the runtime state with a mode $m \in \{\text{use}, \text{edit}\}$. A gesture g (with $e = \text{recognize}(g)$, n the routed node) dispatches on m : in **use** it is the runtime step $\sigma(n) \mapsto \text{handle}_{\tau(n)}(\sigma(n), e)$; in **edit** it is an authoring operation on $\ell(n)$, permitted iff its kind lies in $\kappa(n)$, with Ψ re-established. The **escape** toggles m , altering neither W nor σ .

Proposition 2.17 (Carry-over). *A use-mode state $(W, \sigma, \dots, \text{use})$ and its escaped edit-mode state $(W, \sigma, \dots, \text{edit})$ share (W, σ) and render identically. Any edit issued in edit mode mutates that shared W ; toggling back renders the mutated W . Edit mode is uniform: every node is manipulable, gated by $\kappa(n)$ alone, independently of handle_{τ} — an inert element ($\text{St}(\tau) = 1$) is still editable.*

Proof. Immediate from Definition 2.16: the toggle is identity on (W, σ) , and the edit branch applies an authoring operation to the same W ; capability gating reads $\kappa(n)$ from ρ , which is defined for every node regardless of its handler. $\square$

2.8 The four faces of expressibility

New in this section. the regular tree grammar G_Σ with $L(G_\Sigma)$; the instruction algebra (State-monad actions); decidability of the finite system.

We exhibit the one reachable set $\mathcal{T}_\Sigma$ as a grammar, an instruction algebra, a direct-manipulation editor, and a decidable object.

Face 1: the grammar.

Proposition 2.18 (Regular tree grammar). *The type algebra presents $\mathbb{T}$ as a regular tree grammar G_Σ : each type is a nonterminal; a product $\text{St}(\tau) = \prod_k \text{St}(\tau_k)$ is a production with right-hand side $\tau_1 \cdots \tau_k$; a sum is a set of alternative productions $\tau \rightarrow \tau_k$; a μ is a recursive nonterminal; the unit 1 is a leaf. The schema Σ is the production set, and $L(G_\Sigma)$ is a regular tree language.*

Theorem 2.19 (Grammar = reachable set). *$L(G_\Sigma) = \text{Reach}(W_\emptyset, \rightarrow_\Sigma) = \mathcal{T}_\Sigma$. Decorating each type with its (finite) state, $\hat{G}_\Sigma$ over $\hat{\mathbb{T}} = \bigsqcup_\tau (\{\tau\} \times \text{St}(\tau))$ is still regular and generates the states: $L(\hat{G}_\Sigma)$ cut by Ψ equals $\mathbb{U}_\Sigma^{\leq B}$.*

Proof. ($L(G_\Sigma) \subseteq \mathcal{T}_\Sigma$, $\supseteq$.) A leftmost G_Σ -derivation of a tree t expands nonterminals in preorder; reading each expansion as “mint the child, attach it under the node just expanded, at the next index” yields a build sequence of Theorem 2.7 producing exactly t , and the productions used are the locally Σ -admissible ones, so $t \in \mathcal{T}_\Sigma$. Conversely the preorder build sequence of Theorem 2.7 for $W^* \in \mathcal{T}_\Sigma$ reads off, step by step, a G_Σ -derivation (mint = choose a production for the child’s type; attach-in-preorder = place it on the right-hand side), so $W^* \in L(G_\Sigma)$. The middle equality is Theorem 2.7. Finiteness of each $\text{St}(\tau)$ makes $\hat{G}_\Sigma$ a finite decoration, hence regular; its trees are the labelled-with-state workspaces, and intersecting with the (non-regular but invariant) Ψ gives the valid states $\mathbb{U}_\Sigma^{\leq B}$. $\square$

Face 2: the instruction algebra (quantified change). An **instruction** is a named operator with a binding from a finite parameter profile to values; its meaning is a function $f : B_{\text{valid}} \rightarrow T(R)$ into an effect monad T , total on valid bindings. For the editor, T is the State monad over workspaces and each primitive of Definition 2.6 is one instruction. A **program** $c_1; \dots; c_m$ denotes the composite $f_{c_m} \bullet \cdots \bullet f_{c_1}$ in T .

Proposition 2.20 (State and change are quantified). *Each $s \in \mathbb{U}_\Sigma^{\leq B}$ is a term of the instruction algebra (its build program); each single state change is an instruction; each multi-step change is a program — the composite of the corresponding State-monad actions. The build map from terms to workspaces is onto $\mathcal{T}_\Sigma$.*

Proof. The instruction constructors freely generate a term algebra; assigning each its State-monad action makes workspaces an algebra and the build map the unique homomorphism. Onteness is Theorem 2.7 (every workspace is some build program’s value); a state change is the action of the instruction effecting it; sequencing is composition in T . $\square$

Face 3: direct manipulation. Equip the document with a render $R : \mathcal{T} \rightarrow \mathcal{V}$ and a lift L sending a gesture to an operation, with the WYSIWYG square $R(W \cdot L(W, g)) = g \cdot R(W)$.

Theorem 2.21 (Direct-manipulation completeness). *Assume R injective on editing-relevant structure and L total over the gestures needed to reach any view. Then edit-mode gestures realize every authoring operation, so $\text{Reach}_{\text{edit}} = \mathcal{T}_{\Sigma}$: every Σ -valid UI is buildable by direct manipulation. Restricting permitted operations by κ cuts this to the corresponding invariant subspace.*

Proof. Under the hypotheses L surjects the gestures onto $\mathcal{O}_{\top}$ on each view, and R being injective on editing-relevant structure means distinct editable states have distinct views, so a gesture sequence realizing a build sequence of Theorem 2.7 exists; transporting Theorem 2.7 across R gives reachability of every view. Capability gating removes exactly the gestures whose lifted operation violates the relevant invariant. $\square$

Face 4: decidability.

Corollary 2.22 (Decidability). *$\mathbb{U}_{\Sigma}^{\leq B}$ is finite (Proposition 2.12) and the joint authoring/runtime relation on it is a finite transition system, so reachability, equivalence, and LTL/CTL queries over the UI are decidable.*

Proof. Finiteness is Proposition 2.12; both relations are relations on a finite set; apply Theorem 2.10's decidability to the joint system. $\square$

Theorem 2.23 (Summary: the four faces). *For an expressible UI the following describe one set, $\mathcal{T}_{\Sigma}$ (resp. its decorated form $\mathbb{U}_{\Sigma}^{\leq B}$): the grammar $L(G_{\Sigma})$ (Theorem 2.19); the image of the build homomorphism of the instruction algebra (Proposition 2.20); the direct-manipulation reachable set $\text{Reach}_{\text{edit}}$ (Theorem 2.21); and a finite, decidable transition system (Corollary 2.22). None enlarges expressibility; each re-presents it.*

Proof. Each clause is the cited result; their common right-hand side is $\mathcal{T}_{\Sigma}$ (states $\mathbb{U}_{\Sigma}^{\leq B}$) by Theorem 2.7. $\square$

2.9 Contexts as a fibration

New in this section. the context projection $\text{ctx} : \mathbb{U}_{\Sigma}^{\leq B} \twoheadrightarrow \text{Ctx}$ and its fibers $\mathbb{U}_{\Sigma, x}^{\leq B}$.

A preference need not rank states absolutely; it ranks them *given* what is fixed — the content the user is working with, the task, the device. We model “what is fixed” as a projection and optimize within its fibers.

Definition 2.24 (Context fibration). A **context projection** is a surjection $\text{ctx} : \mathbb{U}_{\Sigma}^{\leq B} \twoheadrightarrow \text{Ctx}$ onto a finite set of **contexts**, with $\text{ctx}(s)$ the fixed part of s . Its **fibers** are $\mathbb{U}_{\Sigma, x}^{\leq B} := \text{ctx}^{-1}(x)$, the states agreeing on the fixed part. (Choosing ctx to forget exactly the user-mutable presentation makes the fibers the “free” part the system may personalize.)

Remark 2.25 (Two readings of context). Reading ctx as an *endogenous* feature of the state (the document content) makes the fibration intrinsic and the ideal a transversal (Section 2.11). Reading context as an *exogenous* signal (time, locale) is the special case where ctx factors through that signal; then a change of context is an event that moves the user to a new fiber, handled by the loop of Section 2.19. We develop the endogenous reading; the exogenous one is a corollary via the loop.

2.10 Preference and its utility representation

New in this section. the preference structure $\{\succeq_x\}_x$ (a total preorder per fiber) and the utility U_u .

Definition 2.26 (Preference structure). A **preference structure** for user u is a family $\{\succeq_x\}_{x \in \text{Ctx}}$ where each $\succeq_x$ is a *total preorder* (reflexive, transitive, total) on the fiber $\mathbb{U}_{\Sigma, x}^{\leq B}$. Write $\succ_x$ and $\sim_x$ for its strict and indifference parts.

The content of the coherence hypothesis is exactly these axioms; the utility is then not an assumption but a theorem.

Theorem 2.27 (Utility representation). *For each total preorder $\succeq_x$ on the finite set $\mathbb{U}_{\Sigma, x}^{\leq B}$ there is a utility $U_u(x, \cdot) : \mathbb{U}_{\Sigma, x}^{\leq B} \rightarrow \mathbb{R}$ with*

$$s \succeq_x s' \iff U_u(x, s) \geq U_u(x, s') \quad (s, s' \in \mathbb{U}_{\Sigma, x}^{\leq B}).$$

U_u is unique up to a per-fiber strictly increasing transformation.

Proof. Let $\sim_x$ be the indifference relation ($s \sim_x s'$ iff $s \succeq_x s'$ and $s' \succeq_x s$); by reflexivity, transitivity, and totality it is an equivalence, and $\succeq_x$ descends to a total *order* (antisymmetric total preorder) on the finite quotient $\mathbb{U}_{\Sigma, x}^{\leq B} / \sim_x$. A finite totally ordered set $\{[s_1] \prec \dots \prec [s_m]\}$ embeds order-preservingly into $\mathbb{R}$ by $[s_i] \mapsto i$. Set $U_u(x, s) := i$ for $s \in [s_i]$. Then $s \succeq_x s'$ iff $[s] \succeq [s']$ iff their indices compare, iff $U_u(x, s) \geq U_u(x, s')$. Any strictly increasing reparametrization of the indices represents the same order, and any representing utility induces the same order, giving uniqueness up to such a transformation. $\square$

Assumption 2.28 (Coherence). u 's preference is a preference structure (Definition 2.26) and is *stationary* over the horizon considered. Equivalently, by Theorem 2.27, a fixed utility $U_u : \{(x, s) : s \in \mathbb{U}_{\Sigma, x}^{\leq B}\} \rightarrow \mathbb{R}$ exists.

Coherence is a genuine restriction — real preferences can be intransitive, fiber-entangled, or drifting. It is the analogue here of the R -injectivity hypothesis of Theorem 2.21: stated, not assumed away.

2.11 The ideal policy and the optimal transversal

New in this section. the ideal policy π_u^* and the optimal transversal $\mathbb{U}^u = \text{im}(\pi_u^*)$.

Definition 2.29 (Ideal policy; optimal state space). Under Assumption 2.28, an **ideal policy** is any $\pi_u^* : \text{Ctx} \rightarrow \mathbb{U}_{\Sigma}^{\leq B}$ with

$$\pi_u^*(x) \in \arg \max_{s \in \mathbb{U}_{\Sigma, x}^{\leq B}} U_u(x, s) \quad (\text{ties broken by a fixed order on } \mathbb{U}_{\Sigma}^{\leq B}).$$

Its graph $\mathbb{U}^u := \text{im}(\pi_u^*) = \{\pi_u^*(x) : x \in \text{Ctx}\}$ is the **optimal state space**.

Proposition 2.30 (The ideal exists, and is a transversal). π_u^* exists, $\text{ctx}(\pi_u^*(x)) = x$, and $\mathbb{U}^u$ meets each fiber in exactly one state: it is a transversal of the fibration ctx , with $|\mathbb{U}^u| = |\text{Ctx}|$.

Proof. Each fiber $\mathbb{U}_{\Sigma, x}^{\leq B}$ is finite (a subset of the finite $\mathbb{U}_{\Sigma}^{\leq B}$, Proposition 2.12) and nonempty (since ctx is onto), so $U_u(x, \cdot)$ attains its maximum on it and the (tie-broken) maximizer $\pi_u^*(x)$ is a well-defined element of $\mathbb{U}_{\Sigma, x}^{\leq B}$; hence $\text{ctx}(\pi_u^*(x)) = x$. Thus $\pi_u^*(x)$ is the unique member of $\mathbb{U}^u$ in fiber x , and distinct contexts give distinct fibers, so $|\mathbb{U}^u| = |\text{Ctx}|$. $\square$

That “an ideal state exists” is thus *finiteness* (Proposition 2.12), not idealization; coherence (Assumption 2.28) only makes the ideal *well-defined*.

2.12 The invariant lattice, precisely

New in this section. invariants as the lattice $2^{\mathbb{U}_{\Sigma}^{\leq B}}$; the induced sub-LTS $S(\phi)$ and the meet-semilattice homomorphism.

Schema, protection, no-dangling, and (below) preference are all invariants of one lattice; restricting the editor by any of them is taking the induced sub-LTS. We make the “lattice” exact.

Definition 2.31 (Invariants and induced subsystems). An **invariant** is a predicate ϕ on $\mathbb{U}_{\Sigma}^{\leq B}$, identified with the set $\llbracket \phi \rrbracket \subseteq \mathbb{U}_{\Sigma}^{\leq B}$. Invariants are ordered by implication $\phi \leq \psi \iff \llbracket \phi \rrbracket \subseteq \llbracket \psi \rrbracket$; this is the complete lattice $(2^{\mathbb{U}_{\Sigma}^{\leq B}}, \subseteq)$ with meet $\wedge = \cap$, join $\vee = \cup$, top $\top = \mathbb{U}_{\Sigma}^{\leq B}$. For an LTS $(\mathbb{U}_{\Sigma}^{\leq B}, \rightarrow)$ let $S(\phi) := (\mathbb{U}_{\Sigma}^{\leq B}, \rightarrow) \upharpoonright_{\llbracket \phi \rrbracket}$ be the induced sub-LTS; order sub-LTSs by componentwise inclusion of states and edges, with meet $\sqcap$ given by intersection of state sets and edge sets.

Theorem 2.32 (Restriction is a meet-semilattice homomorphism). *S is monotone and preserves arbitrary meets: $S(\bigwedge_i \phi_i) = \bigcap_i S(\phi_i)$. In particular the editor constrained by two invariants is the meet of the two restricted editors. S need not preserve joins.*

Proof. Monotone: if $\llbracket \phi \rrbracket \subseteq \llbracket \psi \rrbracket$ then $\llbracket \phi \rrbracket$ -states are $\llbracket \psi \rrbracket$ -states and edges within $\llbracket \phi \rrbracket$ are edges within $\llbracket \psi \rrbracket$, so $S(\phi)$ is a sub-LTS of $S(\psi)$. *Meets:* write $P_i = \llbracket \phi_i \rrbracket$ and $P = \bigcap_i P_i$. The states of $S(\bigwedge_i \phi_i)$ are P ; its edges are $\rightarrow \cap (P \times P)$. The states of $\bigcap_i S(\phi_i)$ are $\bigcap_i P_i = P$; its edges are $\bigcap_i (\rightarrow \cap (P_i \times P_i)) = \rightarrow \cap \bigcap_i (P_i \times P_i) = \rightarrow \cap (P \times P)$, using $\bigcap_i (P_i \times P_i) = (\bigcap_i P_i) \times (\bigcap_i P_i)$ (a pair lies in every $P_i \times P_i$ iff both coordinates lie in every P_i). The two subsystems coincide. *Joins fail:* for $P_1 \neq P_2$ an edge between a P_1 -only and a P_2 -only state lies in $\rightarrow \cap ((P_1 \cup P_2)^2)$ but in neither $\rightarrow \cap (P_i^2)$, so $S(\phi_1 \vee \phi_2) \neq S(\phi_1) \sqcup S(\phi_2)$ in general. $\square$

Example 2.33 (The shipped invariants). The schema Σ , the protection invariant Φ_P^θ (a fixed set of nodes kept in the document with frozen types), and the no-dangling Ψ are invariants; the editor one ships is the meet $\Sigma \wedge \Psi \wedge \Phi_P^\theta$ and its state space is S of that meet. The ideal editor is the top $\top$.

2.13 Personalization as a learned invariant

New in this section. the preference invariant Φ_u and the personalized configuration.

Definition 2.34 (Preference invariant; personalized configuration). The **preference invariant** pins a state to the optimum for its own context:

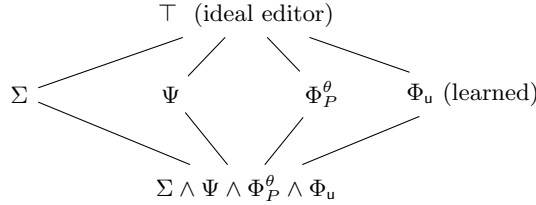
$$\Phi_u(s) :\iff s = \pi_u^*(\text{ctx}(s)).$$

The **personalized configuration** is $S(\Phi_u)$ on $\Sigma \wedge \Psi \wedge \Phi_P^\theta$, i.e. the induced subsystem of the meet $\Sigma \wedge \Psi \wedge \Phi_P^\theta \wedge \Phi_u$.

Proposition 2.35 (The personalized state space is the optimal transversal). $\llbracket \Phi_u \rrbracket = \mathbb{U}^u$, and on it every realized state is a U_u -maximizer for its context: $U_u(\text{ctx}(s), s) = \max_{s' \in \mathbb{U}_{\Sigma, \text{ctx}(s)}^{\leq \mathbf{B}}} U_u(\text{ctx}(s), s')$ for all $s \in \mathbb{U}^u$. By Theorem 2.32 the personalized configuration is one more meet in the lattice of Example 2.33; it is a restriction and need not be strongly connected.

Proof. $s \in \llbracket \Phi_u \rrbracket$ iff $s = \pi_u^*(\text{ctx}(s))$, i.e. s is the fiber- $\text{ctx}(s)$ optimum; by Proposition 2.30 these are exactly the transversal points $\mathbb{U}^u$, on which the maximizer property holds by Definition 2.29. The meet statement is Theorem 2.32 applied to $\Sigma \wedge \Psi \wedge \Phi_P^\theta \wedge \Phi_u$. Strong connectivity is not claimed: a transversal can have any internal edge structure, including none. $\square$

What *is* guaranteed — and is all the agent needs — is that the optimum is reachable *from default* in the unrestricted editor, by strong connectivity (Theorem 2.7).



2.14 The behavioural observation model

New in this section. actions a (interactions and edits); the Boltzmann/Gibbs behavioural law $\mathbb{P}_\theta(a \mid s)$ with rationality β .

The data are not annotations but the user’s own steps in the two transition systems. An action reveals a noisy preference for where the user steered the UI.

Definition 2.36 (Action, behavioural law). At a UI state s (context $x = \text{ctx}(s)$) the user takes an **action** a — a κ -permitted instruction (an *explicit modification*, a step of $\rightarrow$) or a runtime event (a *normal interaction*, a step of $\Rightarrow$) — reaching $s' = a \cdot s$. Let $\mathcal{A}(s)$ be the finite set of available actions. Given a candidate utility U_θ , the **behavioural law** is the Gibbs (Boltzmann-rational) choice

$$\mathbb{P}_\theta(a \mid s) = \frac{\exp(\beta U_\theta(\text{ctx}(a \cdot s), a \cdot s))}{\sum_{a' \in \mathcal{A}(s)} \exp(\beta U_\theta(\text{ctx}(a' \cdot s), a' \cdot s))}, \quad \beta > 0,$$

so higher-utility destinations are more probable, deterministically optimal as $\beta \rightarrow \infty$. The likelihood of a trace $\langle (s_i, a_i) \rangle_{i=1}^t$ is $\prod_{i=1}^t \mathbb{P}_\theta(a_i \mid s_i)$ (Markov).

Both transition systems feed one likelihood: an edit and an interaction are both actions, differing only in which destinations they make available. This is the precise sense in which the model “updates on both normal interactions and explicit modifications.”

2.15 The usage model and the Bayes update

New in this section. the finite hypothesis class Θ and prior μ_0 ; the posterior (usage model) μ_t , the Bayes operator learn , and the MAP estimate $\hat{\theta}_t$.

Definition 2.37 (Hypothesis class, usage model, learning). Fix a finite **hypothesis class** Θ of candidate utilities (the “weights”), and a prior $\mu_0 \in \Delta(\Theta)$. The **usage model** at time t is the posterior $\mu_t \in \Delta(\Theta)$. **Learning** is the Bayes operator

$$\mu_{t+1} = \text{learn}(\mu_t, (s_t, a_t)) \propto \mu_t(\cdot) \mathbb{P}_{(\cdot)}(a_t \mid s_t),$$

a function of interaction history with *no* side effect on any workspace (the descriptive “wall”). The **current weights** are μ_t (or its mode $\hat{\theta}_t = \arg \max_{\theta} \mu_t(\theta)$).

A finite Θ keeps the update exact and every downstream quantity computable; a continuous Θ is the practical case but is not needed for the results below.

Definition 2.38 (Ideal weights; alignment). The **ideal weights** $\theta^* \in \Theta$ are those representing the true preference (Assumption 2.39). The current and ideal weights are two distinct points of Θ — the realized estimate and the oracle target — related by the **alignment operator** $\Lambda = \lim_t \text{learn}(\cdot; \text{history}_{\leq t})$, the Bayes update iterated to its limit.

2.16 Consistency: the current weights reach the ideal

New in this section. realizability and the ideal weights θ^* ; the alignment operator Λ ; the divergence D_{KL} .

Assumption 2.39 (Realizability). The true behavioural law is $\mathbb{P}_{\theta^*}$ for some $\theta^* \in \Theta$, and $\mu_0(\theta^*) > 0$.

Assumption 2.40 (Identifiability and exploration). For every $\theta \neq \theta^*$ there is a state s reachable infinitely often under the data-generating process with $\mathbb{P}_\theta(\cdot | s) \neq \mathbb{P}_{\theta^*}(\cdot | s)$; and at each such s the realized actions are drawn from $\mathbb{P}_{\theta^*}(\cdot | s)$ independently across visits. (Distinct utilities are behaviourally distinguishable, and the distinguishing situations recur.)

Theorem 2.41 (Posterior consistency / alignment). *Under Assumptions 2.39 and 2.40, $\mu_t(\theta^*) \rightarrow 1$ almost surely; hence $\hat{\theta}_t = \theta^*$ for all large t , and the alignment operator Λ carries any admissible prior to the point mass δ_{θ^*} .*

Proof. Fix $\theta \neq \theta^*$. From the Bayes recursion the posterior odds factor:

$$\frac{\mu_t(\theta)}{\mu_t(\theta^*)} = \frac{\mu_0(\theta)}{\mu_0(\theta^*)} \prod_{i=1}^t \frac{\mathbb{P}_\theta(a_i | s_i)}{\mathbb{P}_{\theta^*}(a_i | s_i)},$$

so $\log \frac{\mu_t(\theta)}{\mu_t(\theta^*)} = \log \frac{\mu_0(\theta)}{\mu_0(\theta^*)} + \sum_{i=1}^t Z_i$ with $Z_i = \log \frac{\mathbb{P}_\theta(a_i | s_i)}{\mathbb{P}_{\theta^*}(a_i | s_i)}$. Group the steps by the state visited. At any state s with $\mathbb{P}_\theta(\cdot | s) \neq \mathbb{P}_{\theta^*}(\cdot | s)$, the per-visit conditional expectation is $\mathbb{E}_{\theta^*}[Z | s] = -D_{\text{KL}}(\mathbb{P}_{\theta^*}(\cdot | s) \| \mathbb{P}_\theta(\cdot | s)) < 0$, strictly, since the laws differ and KL vanishes only for identical laws. By Assumption 2.40 at least one such s is visited infinitely often with i.i.d. draws, so along that subsequence the strong law of large numbers makes the running average of Z converge to a strictly negative constant; the contributions of states where the laws agree are identically zero. Hence $\sum_{i \leq t} Z_i \rightarrow -\infty$ a.s., so the odds $\frac{\mu_t(\theta)}{\mu_t(\theta^*)} \rightarrow 0$, i.e. $\mu_t(\theta) \rightarrow 0$. As Θ is finite and the masses sum to one, $\mu_t(\theta^*) \rightarrow 1$ a.s. Consequently, for large t , $\mu_t(\theta^*) > \mu_t(\theta)$ for every $\theta \neq \theta^*$, so $\hat{\theta}_t = \theta^*$; and Λ , the limit of the update, sends the prior to δ_{θ^*} . $\square$

Remark 2.42 (Misspecification). If $\theta^* \notin \Theta$ (no candidate is exactly right), the same odds computation, with $\mathbb{E}[Z | s] = D_{\text{KL}}(\mathbb{P}_{\text{true}} \| \mathbb{P}_{\theta'}) - D_{\text{KL}}(\mathbb{P}_{\text{true}} \| \mathbb{P}_\theta)$, shows the posterior concentrates on the information projection $\theta^\dagger = \arg \min_{\theta \in \Theta} \sum_s w(s) D_{\text{KL}}(\mathbb{P}_{\text{true}}(\cdot | s) \| \mathbb{P}_\theta(\cdot | s))$ (w the visit frequencies): the best-in-class hypothesis. The conclusions below then hold for the best achievable policy rather than the true one.

2.17 The predictive law and the MAP policy

New in this section. the fiber Gibbs law $G_\theta(s \mid x)$ and the MAP policy $\hat{\pi}_\theta$.

The agent needs not the next action but the *destination*: the best state in the current fiber. Utility and probability are one object here.

Definition 2.43 (Fiber Gibbs law; MAP policy). For θ and context x , the **fiber Gibbs law** $G_\theta(s \mid x) \propto \exp(\beta U_\theta(x, s))$ on $\mathbb{U}_{\Sigma, x}^{\leq B}$ is the predictive law over which state should hold. Its **mode** is the **MAP policy** $\hat{\pi}_\theta(x) = \arg \max_{s \in \mathbb{U}_{\Sigma, x}^{\leq B}} G_\theta(s \mid x)$.

Proposition 2.44 (Mode = utility-argmax = ideal at θ^*). *For every $\beta > 0$, $\hat{\pi}_\theta(x) = \arg \max_{s \in \mathbb{U}_{\Sigma, x}^{\leq B}} U_\theta(x, s)$. Hence under Assumptions 2.28 and 2.39, $\hat{\pi}_{\theta^*} = \pi_u^*$, and by Theorem 2.41 $\hat{\pi}_{\hat{\theta}_t} \rightarrow \pi_u^*$. The “highest probability” state of Definition 2.43 and the “ideal” state of Definition 2.29 are the same object; the argmax of either is the ideal.*

Proof. $\exp(\beta \cdot)$ is strictly increasing, so it preserves the maximizer: $\arg \max_s G_\theta(s \mid x) = \arg \max_s U_\theta(x, s)$. At θ^* , $U_{\theta^*} = U_u$ (Assumption 2.39 with Theorem 2.27), so the mode is $\pi_u^*(x)$ by Definition 2.29. Convergence $\hat{\theta}_t \rightarrow \theta^*$ (Theorem 2.41) gives $\hat{\pi}_{\hat{\theta}_t} = \hat{\pi}_{\theta^*} = \pi_u^*$ for large t . $\square$

2.18 The value-driven agent

New in this section. the glamour and grimoire; the value-driven agent F and its value function U ; the idealized UI tuple.

Definition 2.45 (Glamour, grimoire, the agent F). A **glamour** is a typed conditional rewrite (guard, action) with guard a predicate on (node, context) and action a κ -permitted instruction; a **grimoire** is a finite set of glammers. The **agent** F is parameterized by a value function U (its “values”). From U it compiles the grimoire whose glamour for context x has guard $\text{ctx} = x$ and action a κ -permitted program driving the default workspace T_0 toward $\hat{\pi}(x)$ (Definition 2.43); as a **pre-pass** — before the runtime layer takes the stage — F emits that program and applies it, yielding the personalized $\langle T_F, \varrho_F \rangle$. F does not learn.

Theorem 2.46 (Safety, reachability, reversibility). *Under Assumption 2.28 (so $\hat{\pi}$ is defined) and expressibility, the agent F of Definition 2.45 satisfies:*

- (i) (reachability) *the target $\hat{\pi}(\text{ctx}(T_0))$ is reachable from T_0 , so F ’s program exists;*
- (ii) (closure / safety) $\langle T_F, \varrho_F \rangle \in \mathcal{T}_\Sigma$ *with Σ and Ψ intact;*
- (iii) (reversibility) *F ’s effect is undoable and reset-to-default is reachable from any personalized state.*

Proof. (i) By Proposition 2.30 the target is a state of $\mathbb{U}_\Sigma^{\leq \mathbf{B}}$; by Theorem 2.7 (strong connectivity) a sequence of primitives carries T_0 to it, and restricting to κ -permitted primitives still suffices because the target satisfies the capability constraints by construction. (ii) Each κ -permitted instruction preserves Σ (it is a step of the restricted editor, the induced subsystem $\mathbf{S}(\Sigma \wedge \dots)$ of Theorem 2.32) and preserves Ψ (removals re-establish it, Definition 2.9); invariant preservation composes because the meet of invariants is itself an invariant (Theorem 2.32), so the whole program keeps the output in $\mathbf{S}(\Sigma \wedge \Psi \wedge \dots) \subseteq \mathcal{T}_\Sigma$. (iii) The effect is a program of primitives and $(\mathcal{T}, \rightarrow)$ is strongly connected (Theorem 2.7), so $W_\varnothing$ — and any prior state — is reachable from the personalized state; undo is the reverse path. $\square$

Definition 2.47 (The idealized UI). For user u the **idealized UI** is the tuple $\mathcal{I}_u = (U_u, \text{ctx}, \Theta, \text{learn}, \theta^*, \pi_u^*, \Phi_u, F)$: an expressible UI (Part A) with a coherent utility over a context fibration (Assumption 2.28 and Definition 2.24), a usage model and its Bayes update (Definition 2.37), aligned current/ideal weights (Definition 2.38 and Theorem 2.41), the induced ideal policy (Definition 2.29), the learned invariant carving the optimal transversal (Definition 2.34), and the value-driven agent that preemptively casts it (Definition 2.45 and Theorem 2.46).

2.19 The closed loop and non-thrashing

New in this section. the closed loop with confidence gate ϵ ; the installed hypothesis and the non-thrashing argument.

The organs close a loop: F applies the current best policy; the user interacts and corrects; the usage model updates; a new policy results. The danger is *thrashing* — the applied policy oscillating rather than settling. The one engineered quantity is *damping*: do not re-apply on weak evidence.

Definition 2.48 (Loop with a confidence gate). Fix a confidence level $\epsilon \in (0, \frac{1}{2})$. The loop maintains an *installed* hypothesis $\theta^{(t)}$ (initially none). At round t , with posterior μ_t : if some θ has $\mu_t(\theta) \geq 1 - \epsilon$, install it ($\theta^{(t)} = \theta$) and have F cast $\hat{\pi}_\theta$; otherwise keep the current installation. (Installing requires decisive evidence; ϵ is the damping parameter. A switching margin on expected utility is an equivalent gate.)

Theorem 2.49 (Non-thrashing and convergence). *Under Assumptions 2.39 and 2.40 and the gate of Definition 2.48, almost surely the installed policy switches finitely often and is eventually constant, equal to $\pi_\star^*$. Without the gate, the MAP-based policy may switch unboundedly often before settling, even though it still converges.*

Proof. By Theorem 2.41 $\mu_t(\theta^\star) \rightarrow 1$ a.s., so there is a finite (random) time T_0 with $\mu_t(\theta^\star) > 1 - \epsilon$ for all $t \geq T_0$. Since masses sum to one and $\epsilon < \frac{1}{2}$, for $t \geq T_0$ no $\theta \neq \theta^\star$ can reach $1 - \epsilon$ (that would force total mass $> 2(1 - \epsilon) > 1$). Hence after T_0 the only gate-eligible hypothesis is $\theta^\star$: the loop installs $\hat{\pi}_{\theta^\star} = \pi_\star^*$ (Proposition 2.44) at most once more and never switches again. Before T_0 only finitely many rounds elapse (as $T_0 < \infty$ a.s.), bounding the number of switches. For the gateless remark: when μ_t is diffuse the mode $\hat{\theta}_t$ can alternate among near-tied hypotheses, changing $\hat{\pi}_{\hat{\theta}_t}$ each time, so the unbounded-switching behaviour is possible despite eventual convergence; the gate suppresses exactly this transient by requiring decisiveness. $\square$

Remark 2.50 (Damping, and the cost it trades). The gate trades *latency* for *stability*: a larger ϵ commits sooner but risks installing a wrong policy that consistency later overturns; a smaller ϵ never installs prematurely but waits longer. Under misspecification (Remark 2.42) the same proof gives finitely many switches and convergence to the gated best-in-class policy $\hat{\pi}_{\theta^\dagger}$.

2.20 What is computable

Proposition 2.51 (Computability). *With Θ and $\mathbb{U}_{\Sigma}^{\leq B}$ finite: the ideal policy π_a^* , the posterior μ_t and its mode, the MAP policy $\hat{\pi}_\theta$, the agent’s program, and the gate decision are all computable. For any fixed installed policy the UI’s behaviour is a finite transition system, so reachability and LTL/CTL queries about it are decidable.*

Proof. Each quantity is an argmax or exact Bayes update over the finite sets $\mathbb{U}_{\Sigma}^{\leq B}$, $\mathcal{A}(s)$, Θ — finite computations. The agent’s program is the build/teardown sequence of Theorem 2.7, computable from source and target. Under a fixed policy the reachable UI states form a finite system (Corollary 2.22), where the named problems are decidable. $\square$

The continuous-posterior Bayesian loop as a whole is not a finite object; the decidable claims concern the *installed-policy* trajectory and the finite-state UI it drives, which is what one actually verifies.

2.21 Synthesis

The idealized UI is the expressible UI read once more. Its restriction to the user's optimum is one more meet Φ_u in the invariant lattice (Theorem 2.32 and Definition 2.34); the optimal states form a transversal of the context fibration (Proposition 2.30); the agent that installs them is the fairy fed a learned value (Definition 2.45); the model that learns it is a Bayes update over the trajectories of *both* transition systems (Definition 2.37); the predictive law it maximizes and the utility that defines the ideal are one object via the Gibbs presentation (Proposition 2.44); and the loop is kept from thrashing by a single confidence gate (Theorem 2.49). The grammar is the tree's declarative image, edit mode its direct-manipulation image, the usage model its measurable image, and the agent its programmable one; the idealized UI is their composition — a learned invariant, cast preemptively — and not a new power bolted on.

Theorem 2.52 (The idealized UI, assembled). *Let a UI be expressible (Definition 2.14) and let user u satisfy coherence (Assumption 2.28), realizability (Assumption 2.39), and identifiability/exploration (Assumption 2.40). Then:*

- (1) *an ideal policy π_u^* exists and is a transversal of the context fibration (Proposition 2.30);*
- (2) *the usage model's current weights converge to the ideal weights, $\hat{\theta}_t \rightarrow \theta^*$, and the MAP policy to the ideal, $\hat{\pi}_{\hat{\theta}_t} \rightarrow \pi_u^*$ (Theorem 2.41 and Proposition 2.44);*
- (3) *enforcing π_u^* is the meet Φ_u in the invariant lattice, whose state space is the optimal transversal (Definition 2.34 and Proposition 2.35);*
- (4) *the value-driven agent F realizes it as a safe, reversible pre-pass (Theorem 2.46); and*
- (5) *under a confidence gate the closed loop switches finitely often and converges to π_u^* (Theorem 2.49).*

Proof. Each clause is the cited result; together they instantiate Definition 2.47. □

2.22 What is free, what is assumed

New in this section. the three idealizing hypotheses recalled: coherence (H1), realizability (H2), identifiability+exploration (H3).

The development separates two kinds of claim.

Free (theorems from finiteness and expressibility). The existence of the ideal (Proposition 2.30) is finiteness (Proposition 2.12); the safety and reversibility of the agent (Theorem 2.46) are closure under the invariant lattice (Theorem 2.32) and strong connectivity (Theorem 2.7); the grammar, instruction algebra, direct manipulation, and decidability (Theorem 2.23) are the four faces of one reachable set. None needs a hypothesis about the user.

Assumed (the idealizing hypotheses). Learnability of the ideal does need hypotheses, and they are exactly three: coherence (Assumption 2.28) — a transitive, complete, stationary preference, equivalently a fixed utility; realizability (Assumption 2.39) — a model class containing the truth; and identifiability with exploration (Assumption 2.40) — distinct preferences behaviourally distinguishable, and the distinguishing situations recurring. These are the analogue here of the R -injectivity caveat of Theorem 2.21, and each can fail: preferences can be intransitive or drifting (Assumption 2.28), unrepresentable (Assumption 2.39, softened to the best-in-class projection by Remark 2.42), or unidentifiable for want of signal (Assumption 2.40).

The safety margin holds regardless. Because every applied program is a κ -permitted instruction, the worst a wrong inference yields is a *valid-but-suboptimal* UI, one undo away (Theorem 2.46); and since the serialized tree the agent reads may carry untrusted content, structural closure bounds the *blast radius* of a bad proposal, not the *intent* behind it. The descriptive-prescriptive wall (the model never edits, the agent never learns) and the confidence gate (Definition 2.48) are the safety spine. The idealized UI is thus a provable function *into* the optimal transversal — as good as its learned utility, and never a trap.

2.23 The twelve truths, located

Informal truth	Formal statement	Where
G1.1 components / attrs / events	elements; atomic (leaf) vs composite (internal); composition = nesting	Definition 2.4 and Remark 2.5
G1.2 every valid state expressible	generative complete- ness over $\mathbb{U}_{\Sigma}^{\leq \mathbb{B}}$; invalid states ex- cluded by type algebra / Σ / Ψ	Definitions 2.11 and 2.14 and Remark 2.13
G1.3 all tools exposed; no sealed <code>doXY</code>	primitivity of $\rightarrow^*$ over $\mathcal{O}_{\top}$	Proposition 2.8
G1.4 generative grammar / DSL	regular tree grammar, $L(G_{\Sigma}) =$ $\mathcal{T}_{\Sigma}$; in- struction language	Proposition 2.18 and Theorem 2.19
G1.5 states & changes quantified by functions	instruction algebra; State- monad actions; programs; finiteness	Proposition 2.20 and Corollary 2.22
G1.6 an editor reaching any state	structural universal- ity; direct- manipulation complete- ness	Theorems 2.7 and 2.21
G2.1 ideal state / ideal state space	ideal policy π_u^* , optimal transversal $\mathbb{U}^u$; <i>exists for free</i>	Definition 2.29 and Proposition 2.30
G2.2 weight-update from use & edits	Bayes up- date over traces of both $\implies$ and $\rightarrow$	Definitions 2.36 and 2.37
G2.3 two weight sets & a transform between	current μ_t vs ideal θ^* ; alignment Λ ; consistency	Definition 2.38 and Theorem 2.41
G2.4 highest-probability function	fiber Gibbs law; MAP policy $\stackrel{53}{=}$ utility- argmax = ideal	Definition 2.43 and Proposition 2.44